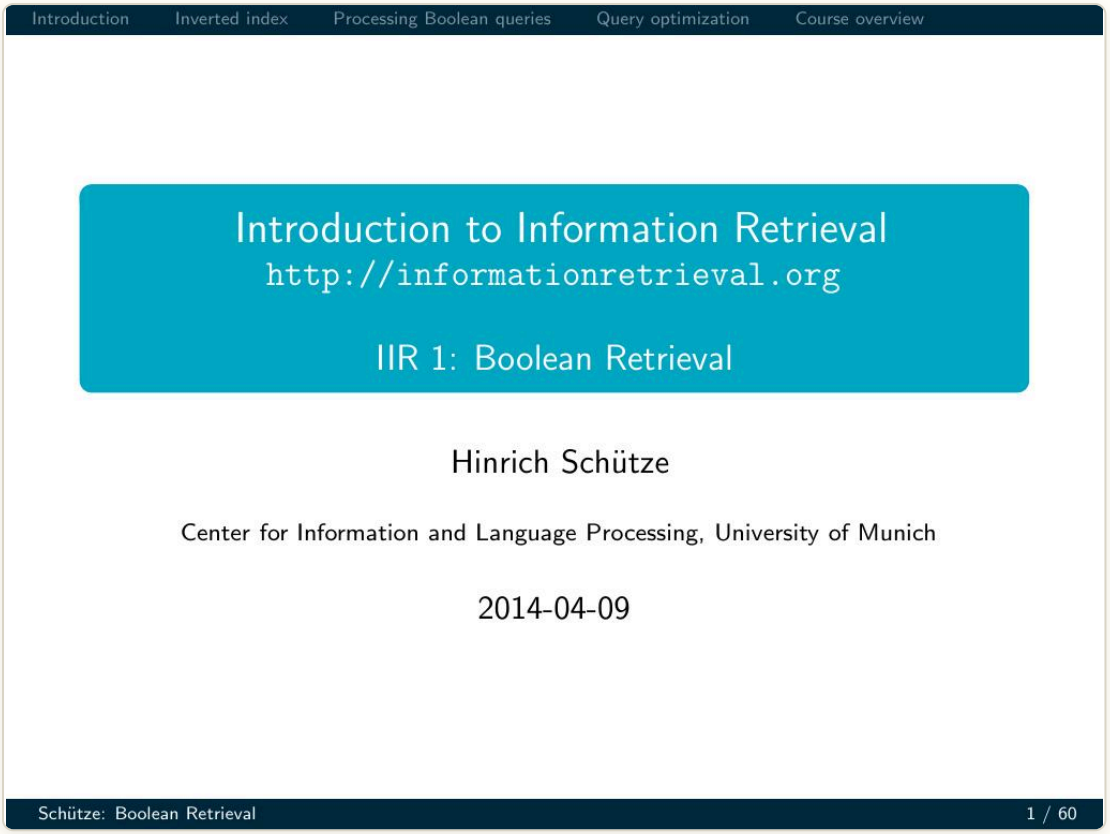
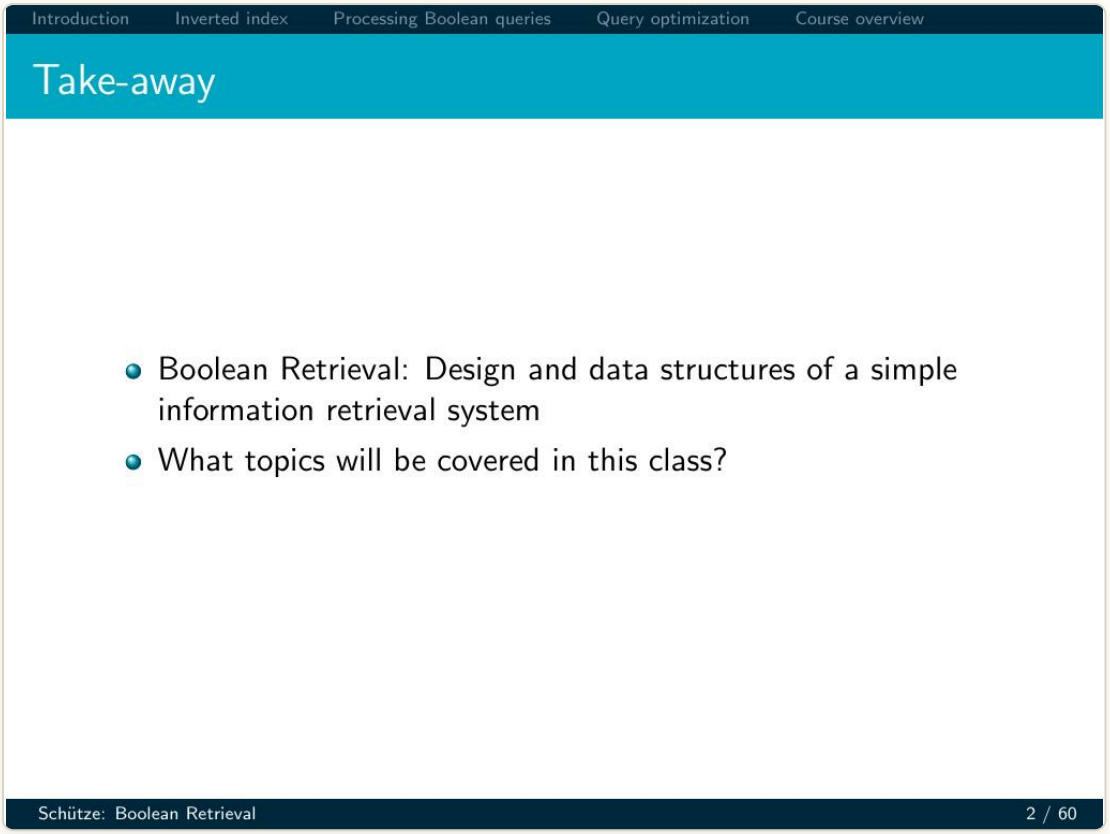


정보검색 입문 — IIR 1: 불리언 검색

- Hinrich Schütze, 뮌헨대 정보·언어처리센터 (2014-04-09)
- 교재: Introduction to Information Retrieval (IIR)

✍️ 메모

💡 이해 1강 핵심: 단어-문서 행렬 → 역색인 → 포스팅 교집합(merge) → 질의 최적화

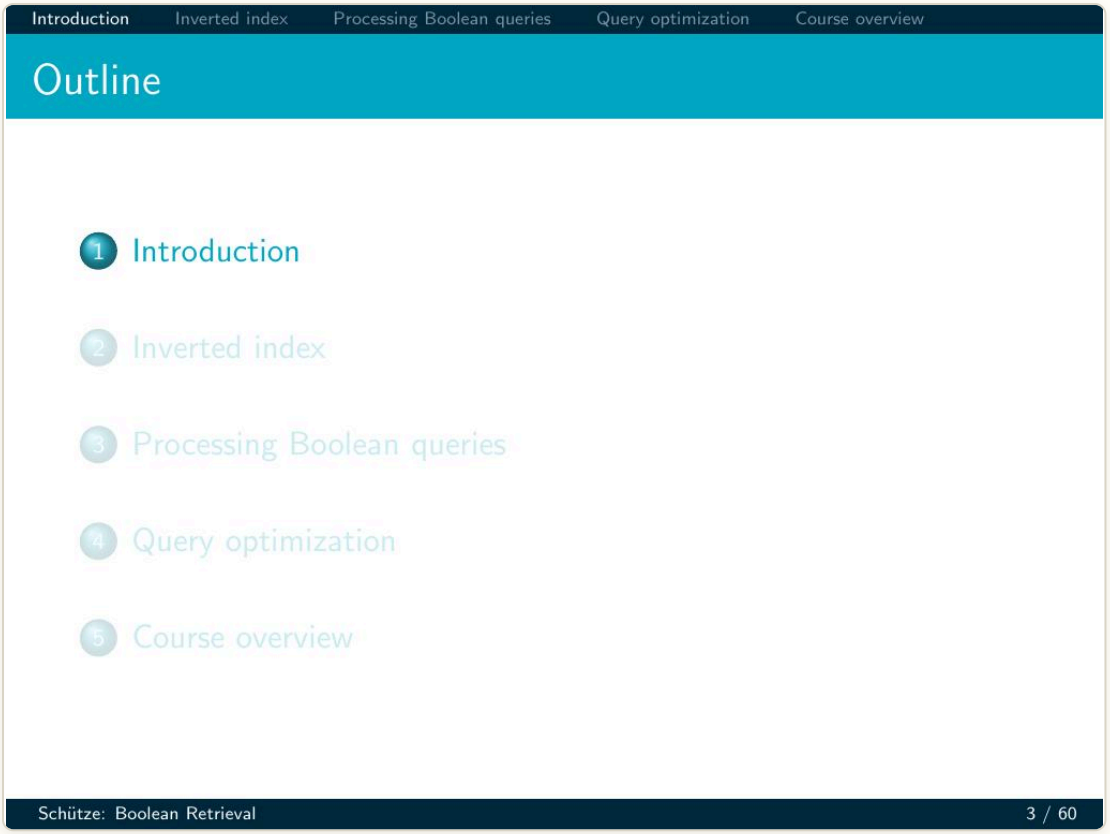


이번 강의에서 가져갈 것

- **불리언 검색**: 간단한 정보검색 시스템의 설계와 자료구조
- 이 수업에서 다룰 주제 미리 보기

✍ 메모

— 넘어가도 되는 쪽

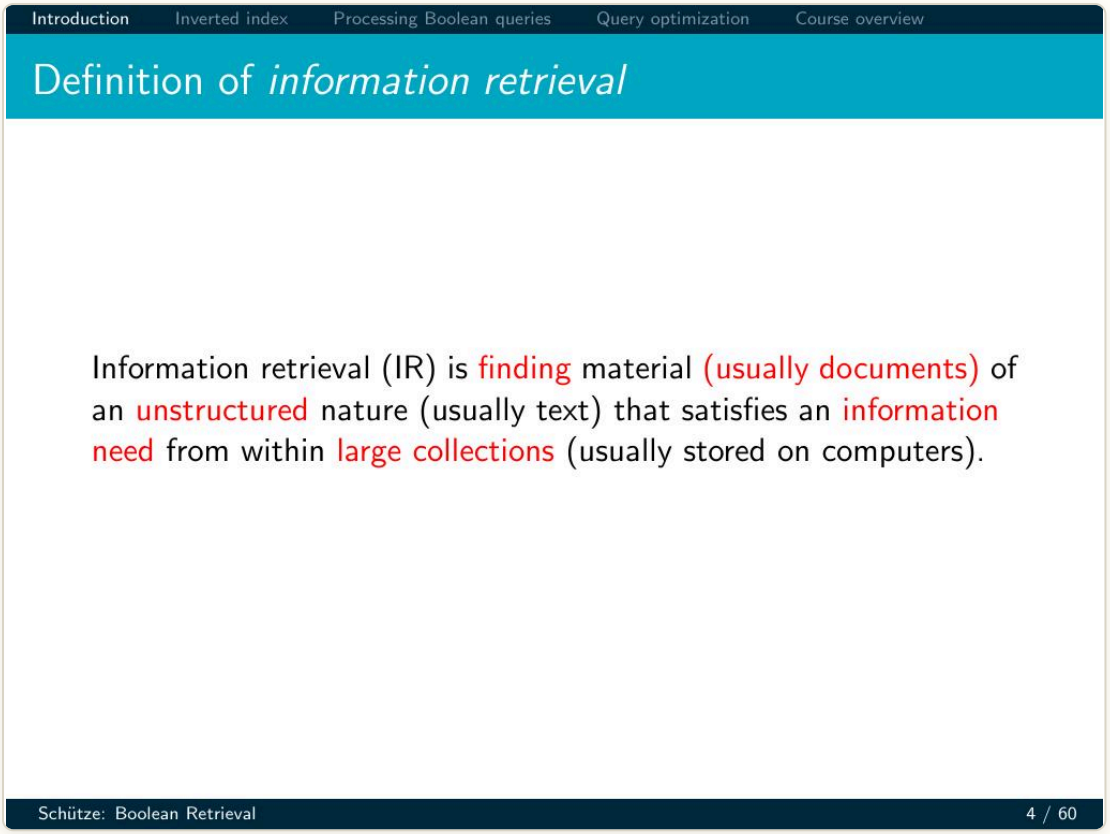


목차

- 소개 → 역색인 → 불리언 질의 처리 → 질의 최적화 → 과정 개관
- 지금: **목차**

✍️ 메모

— 넘어가도 되는 쪽

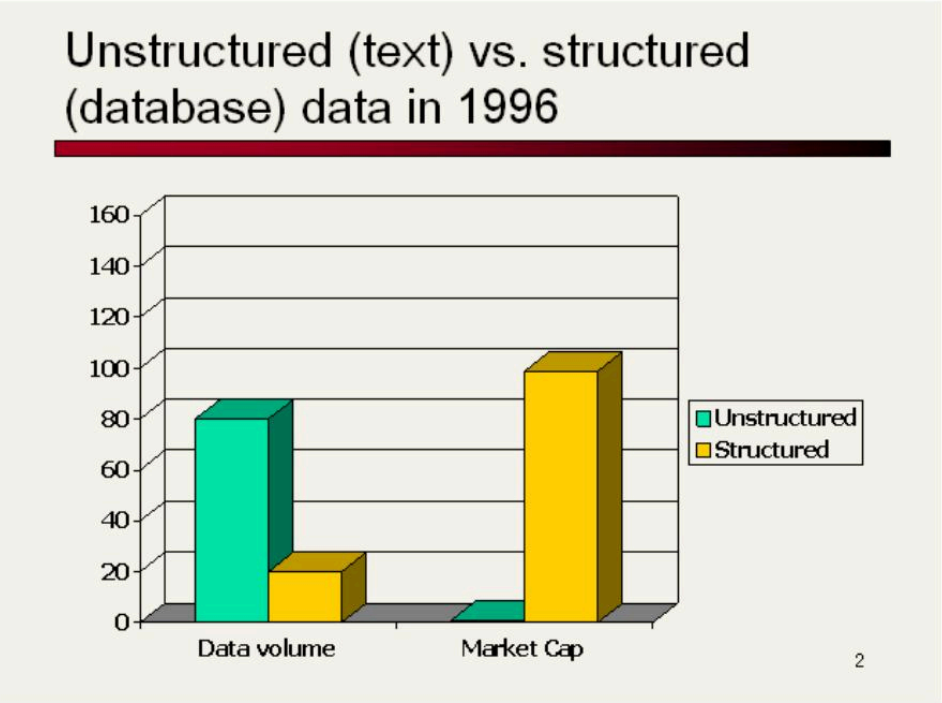


정보검색의 정의

- **정보검색(IR)**은 **대규모 컬렉션**(보통 컴퓨터에 저장)에서 **정보 요구(information need)**를 만족하는
- **비정형(unstructured)** 성격의 자료(보통 **텍스트**)를 — 보통 **문서**를 — 찾는 것

✍ 메모

★ 시험 정의의 키워드 4개: 비정형 · 텍스트/문서 · 정보 요구 · 대규모 컬렉션

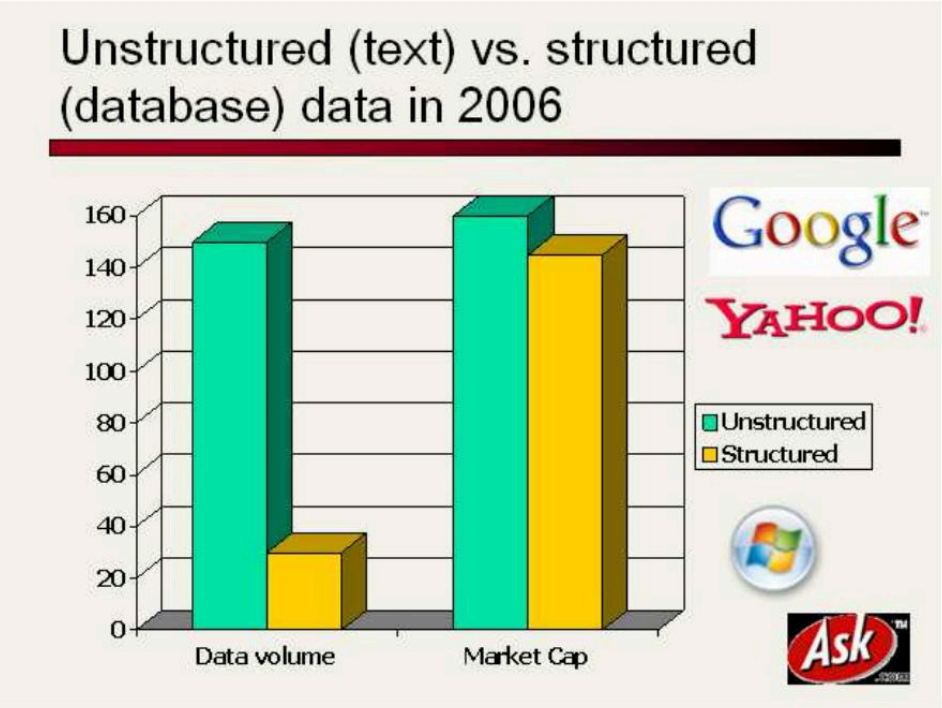


비정형 vs 정형 데이터 (1996년)

- 그래프: **데이터 양**은 비정형(텍스트)이 더 많지만, **시가총액**은 정형(DB) 쪽이 압도적

✍ 메모

— 넘어가도 되는 쪽

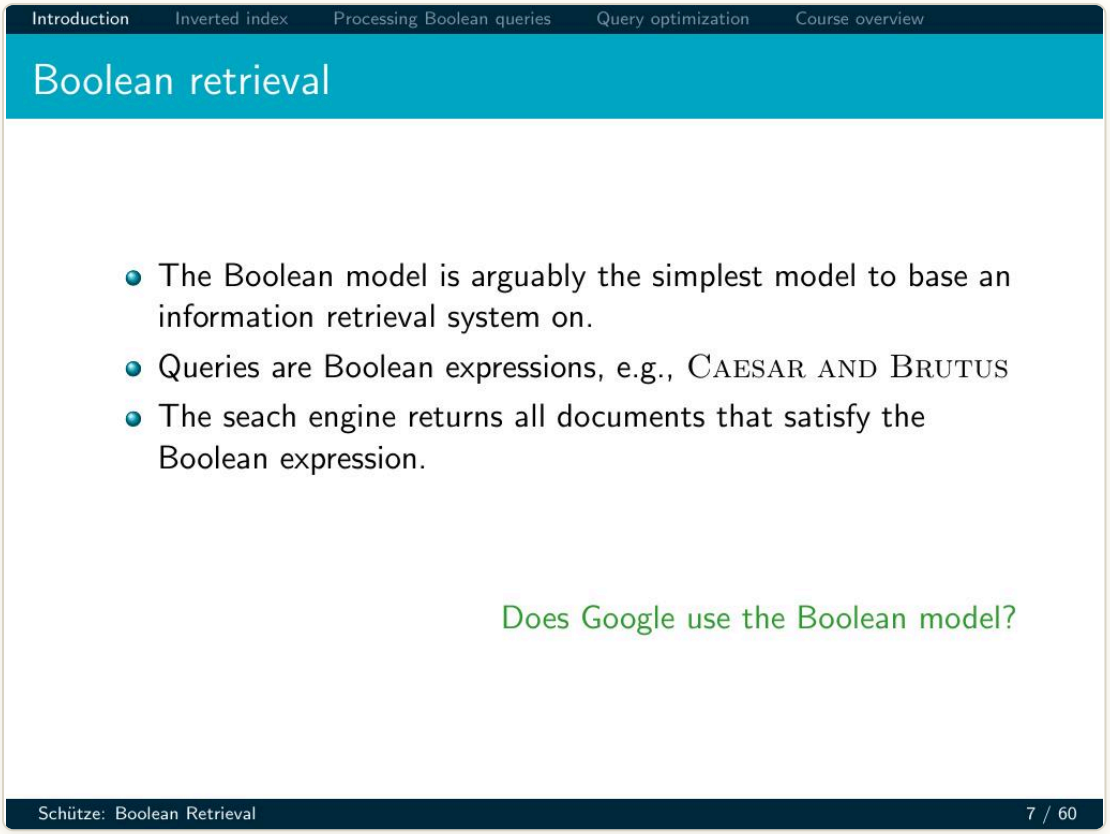


비정형 vs 정형 데이터 (2006년)

- 그래프: 이제 **시가총액도 비정형 쪽이 더 큼** — 구글·야후·MS·Ask 같은 검색 회사가 성장

✍️ 메모

💡 이해 요지: 10년 사이 텍스트 검색이 돈이 되는 시장이 됐다

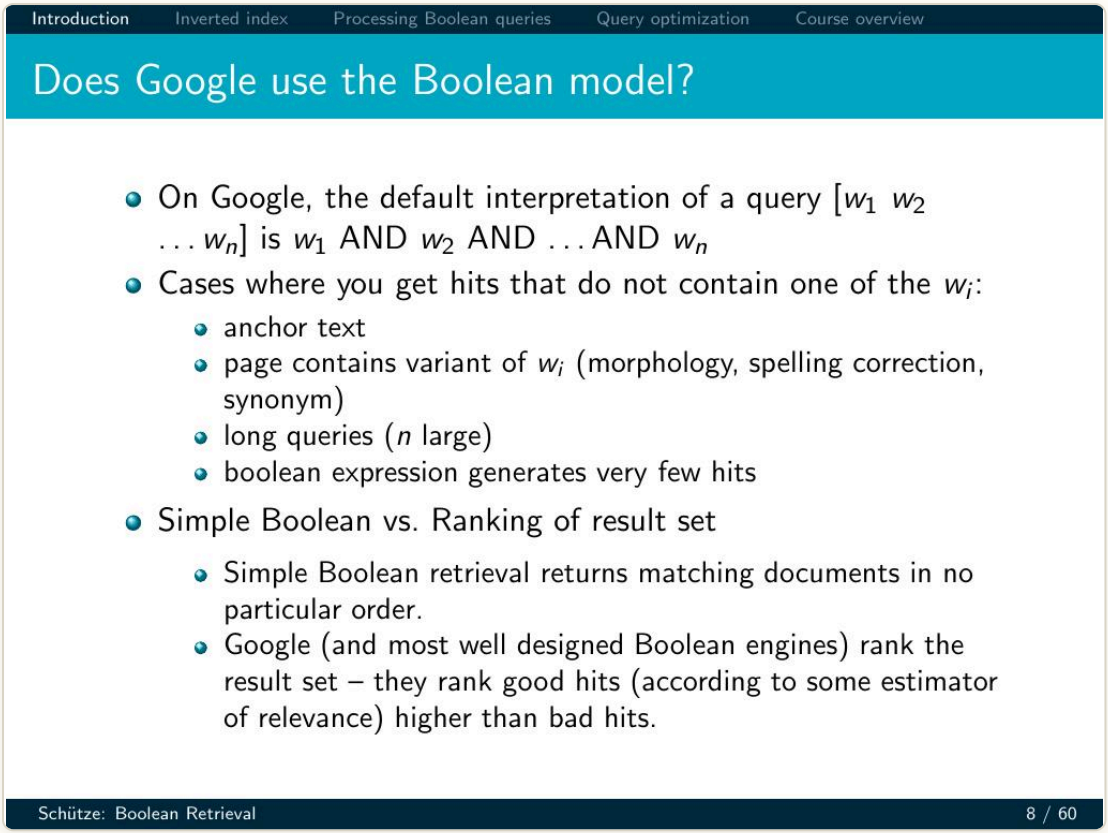


불리언 검색 (Boolean retrieval)

- **불리언 모델**은 IR 시스템의 기반이 될 수 있는 **가장 단순한 모델**이라 할 만하다
- 질의가 **불리언 식** — 예: Caesar AND Brutus
- 검색 엔진은 그 식을 **만족하는 모든 문서**를 돌려준다
- 구글은 불리언 모델을 쓸까?

✍️ 메모

★ 시험 불리언 = 맞다/아니다 두 가지뿐 (순위 없음)

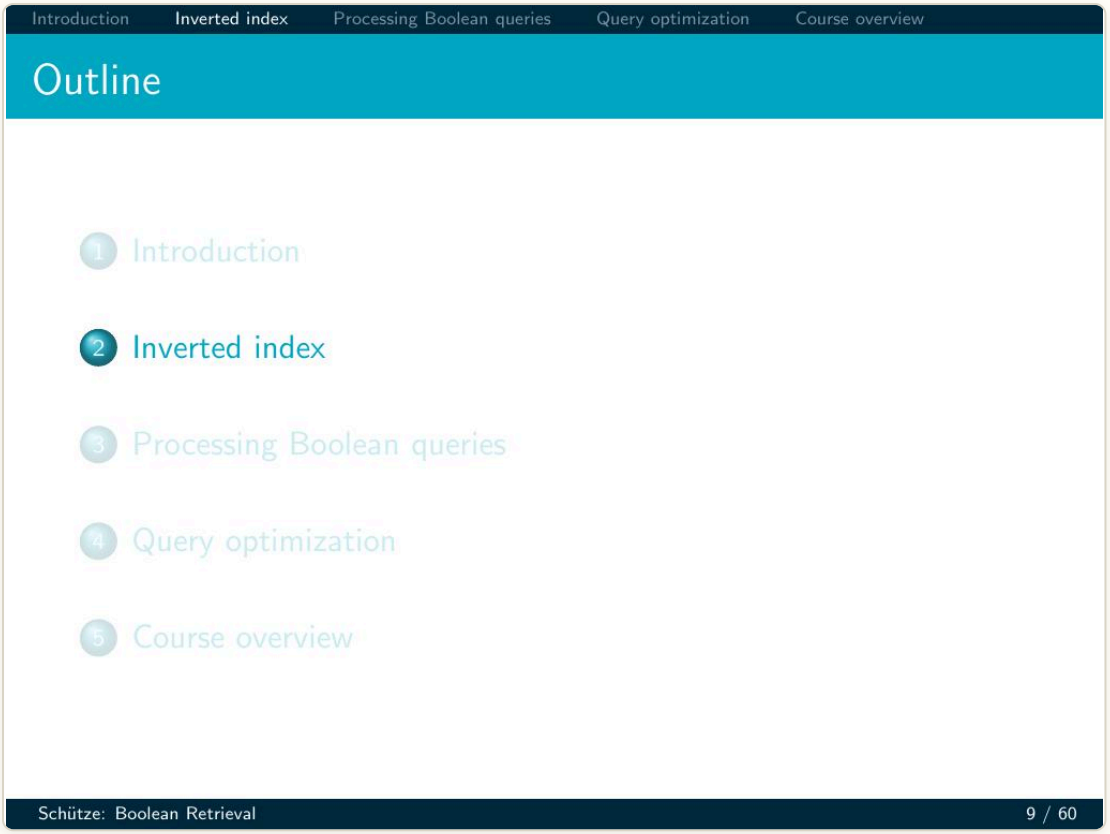


구글은 불리언 모델을 쓰나?

- 구글에서 질의 [w1 w2 ... wn]의 기본 해석은 **w1 AND w2 AND ... AND wn**
- 그런데 어떤 wi가 없는 결과가 나오는 경우
  - **앵커 텍스트**(다른 페이지가 이 페이지를 가리킬 때 쓴 글)
  - 페이지에 wi의 **변형**이 있음 (형태 변화, 철자 교정, 동의어)
  - **긴 질의** (n이 큼)
  - 불리언 식으로는 결과가 **아주 적을** 때
- **단순 불리언 vs 결과 집합의 순위화**
  - 단순 불리언은 맞는 문서를 **특별한 순서 없이** 돌려줌
  - 구글(과 잘 만든 불리언 엔진)은 결과를 **순위화** — 관련성 추정치가 높은 것을 위로

✍️ 메모

⚠ 주의 구글 = AND가 기본이지만 순수 불리언은 아니다 (순위화 + 예외들)

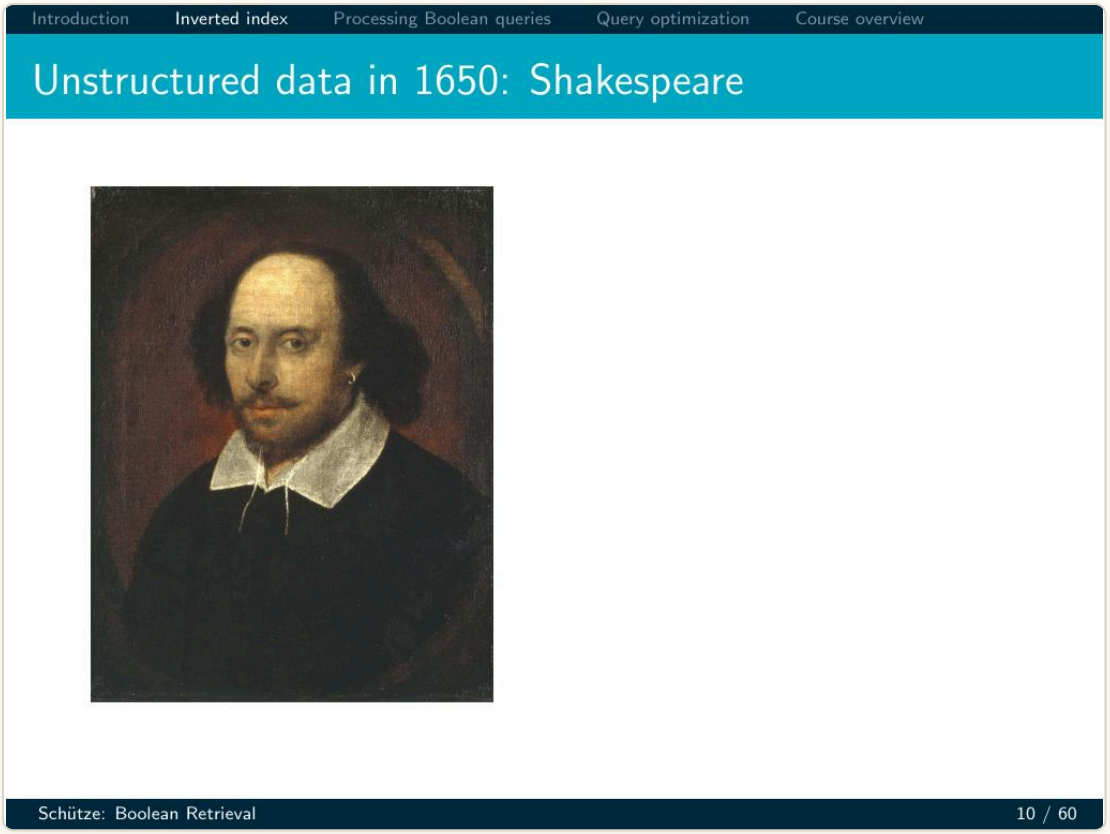


목차

- 소개 → 역색인 → 불리언 질의 처리 → 질의 최적화 → 과정 개관
- 지금: **역색인 (Inverted index)**

✍️ 메모

— 넘어가도 되는 쪽

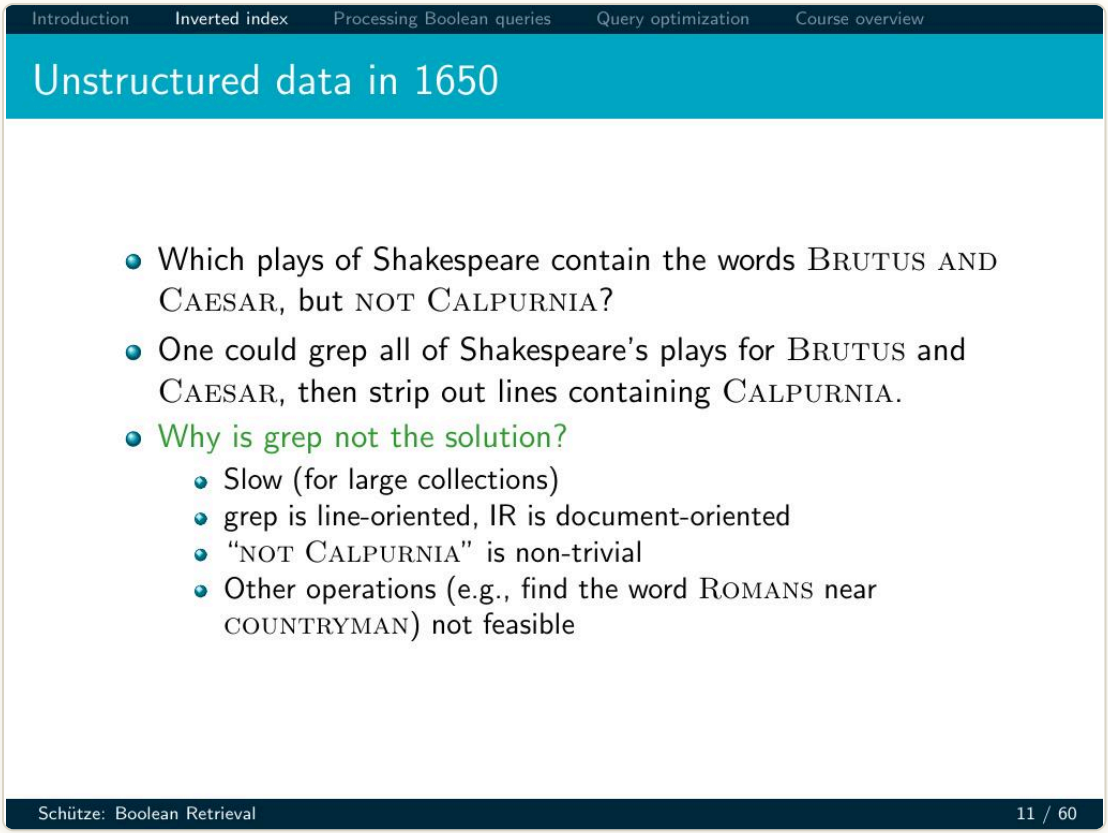


1650년의 비정형 데이터: 셰익스피어

- 셰익스피어 초상 — 이제부터 셰익스피어 희곡 모음을 예제 컬렉션으로 쓴다

✍ 메모

— 넘어가도 되는 쪽

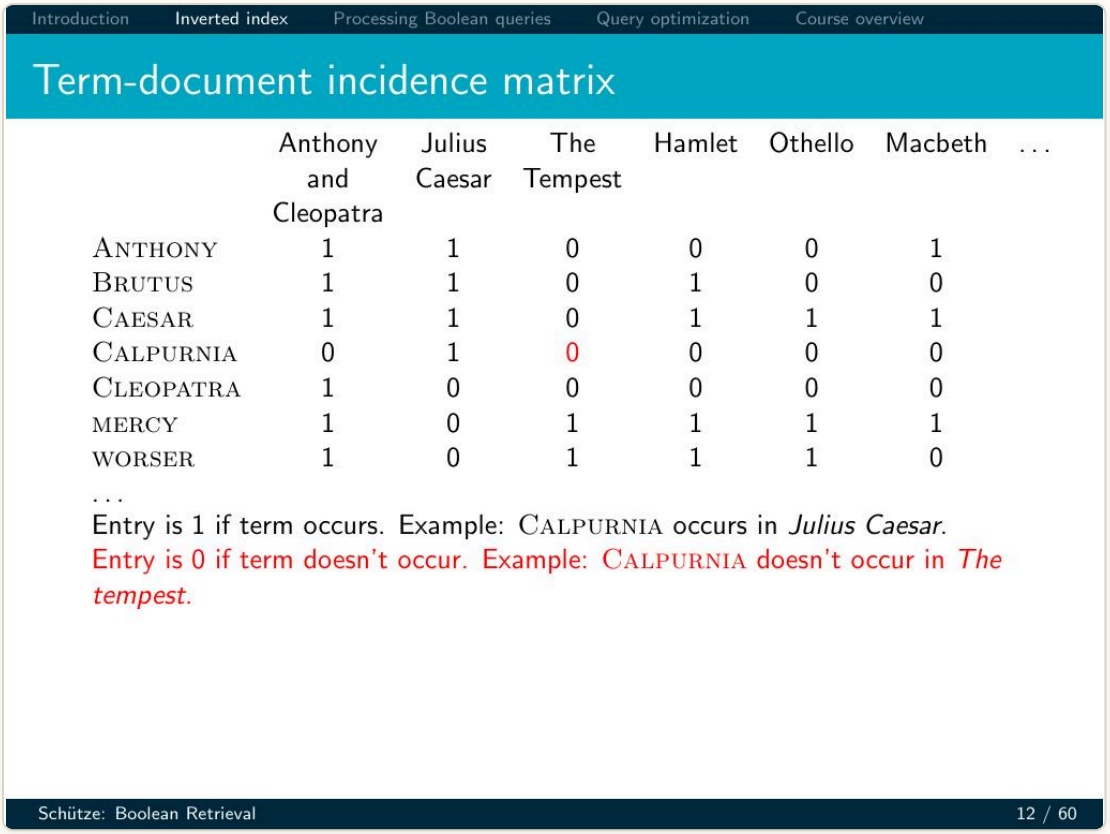


1650년의 비정형 데이터 — 질의 예

- Brutus와 Caesar는 있고 Calpurnia는 **없는** 셰익스피어 희곡은?
- 모든 희곡을 Brutus·Caesar로 **grep**하고 Calpurnia가 있는 줄을 빼면?
- grep이 답이 아닌 이유
  - **느리다** (큰 컬렉션에서)
  - grep은 **줄 단위**, IR은 **문서 단위**
  - 'not Calpurnia'가 간단하지 않다
  - 다른 연산 (예: Romans가 countryman **근처**에 있는 것 찾기)이 불가능

✍ 메모

★ 시험 **grep의 한계 4개: 느림 · 줄 단위 · NOT 어려움 · 근접 검색 불가**

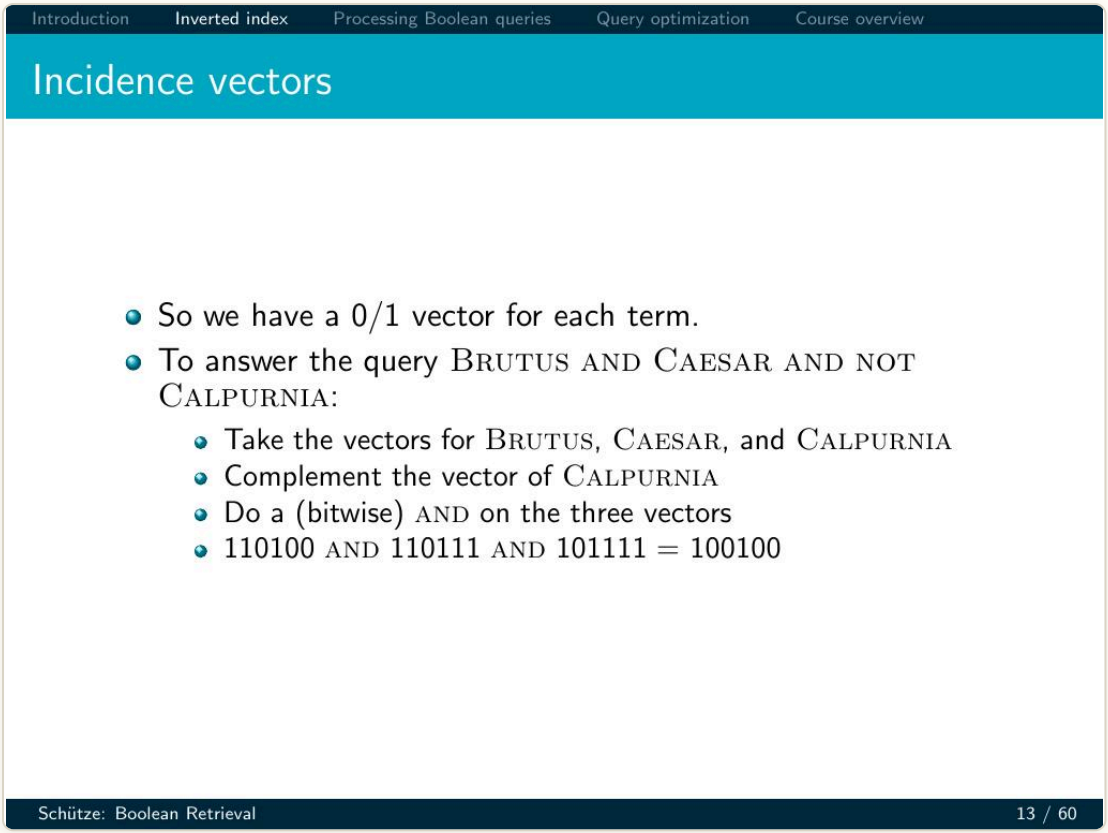


단어-문서 발생 행렬 (Term-document incidence matrix)

- 행 = 단어(Anthony, Brutus, Caesar, Calpurnia, Cleopatra, mercy, worser ...), 열 = 희곡
- 단어가 나오면 **1** — 예: Calpurnia는 Julius Caesar에 나옴
- 안 나오면 **0** — 예: Calpurnia는 The Tempest에 안 나옴

✍ 메모

— 넘어가도 되는 쪽

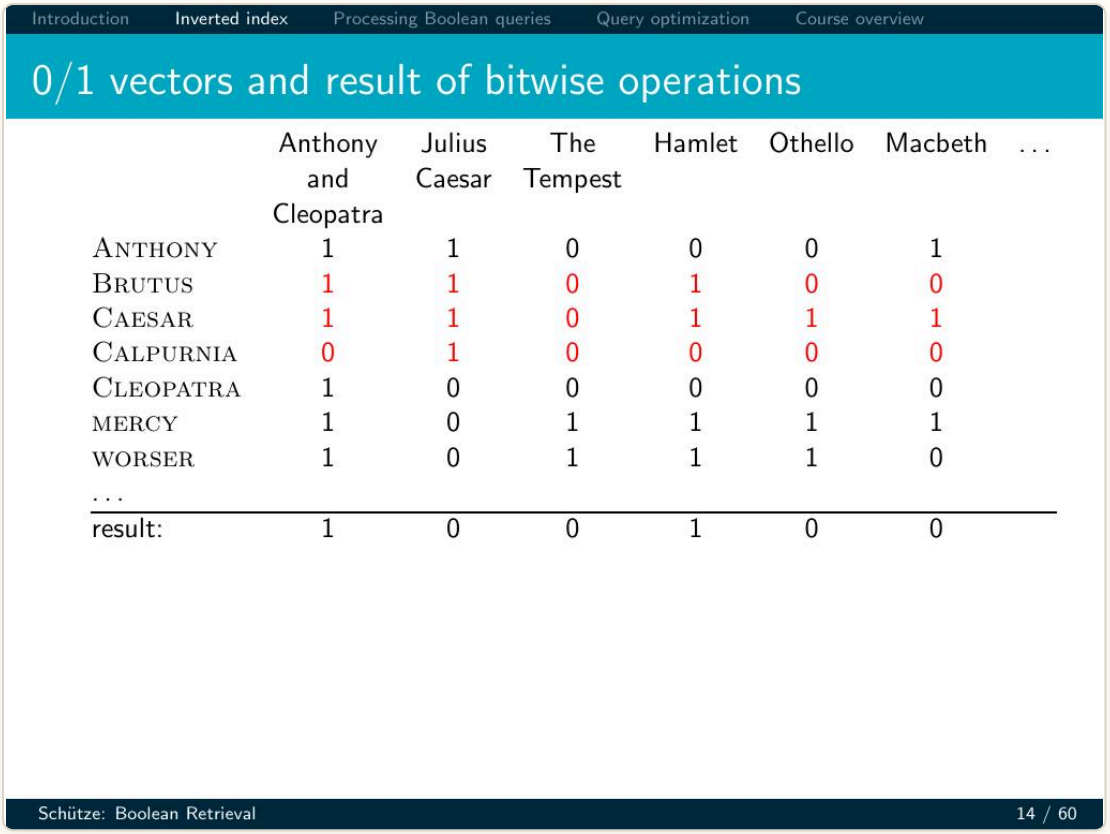


발생 벡터 (Incidence vectors)

- 단어마다 **0/1 벡터**가 생긴다
- Brutus AND Caesar AND NOT Calpurnia 답하기
  - Brutus, Caesar, Calpurnia의 벡터를 꺼낸다
  - Calpurnia 벡터의 **보수(complement)**를 취한다
  - 세 벡터를 **비트 AND**
  - **110100 AND 110111 AND 101111 = 100100**

✍️ 메모

✏ 예·풀이 Calpurnia 010000 → 보수 101111. 결과 100100 = Anthony and Cleopatra, Hamlet

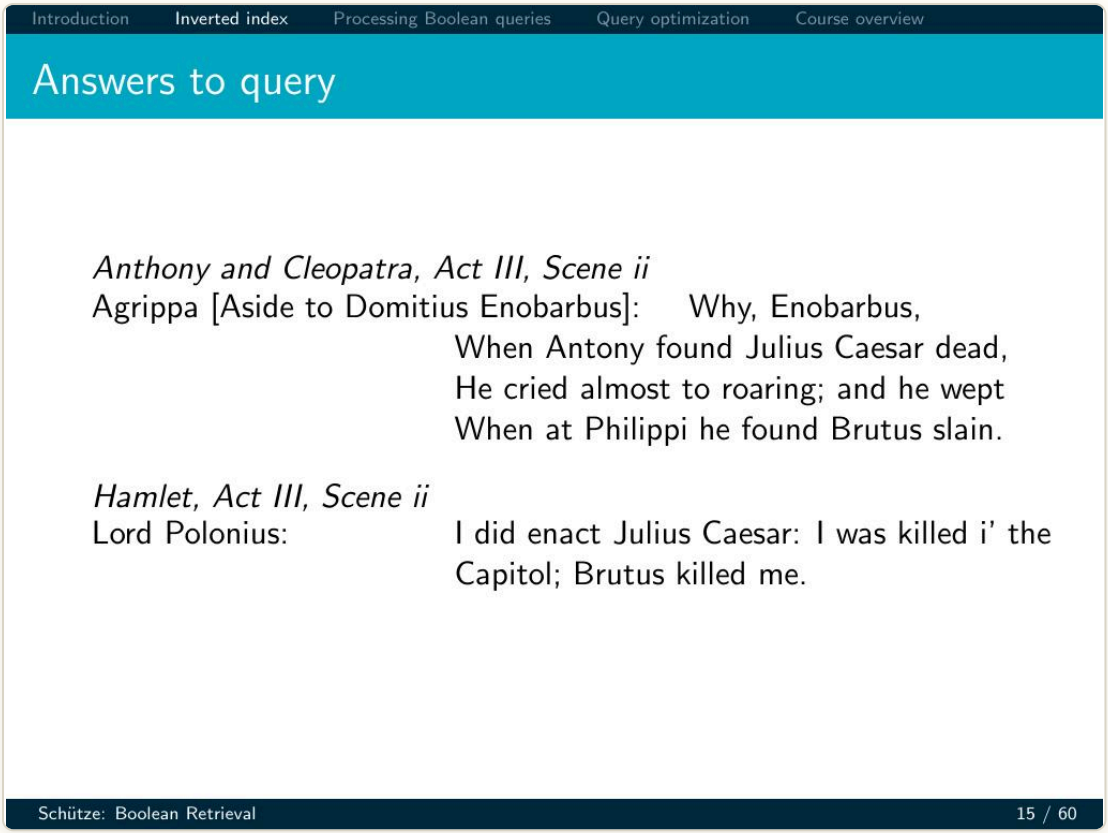


0/1 벡터와 비트 연산 결과 (표)

- Brutus·Caesar·Calpurnia 행을 빨갛게 표시, 결과 행: **1 0 0 1 0 0**
- → 첫째 희곡(Anthony and Cleopatra)과 넷째(Hamlet)

✍ 메모

— 넘어가도 되는 쪽

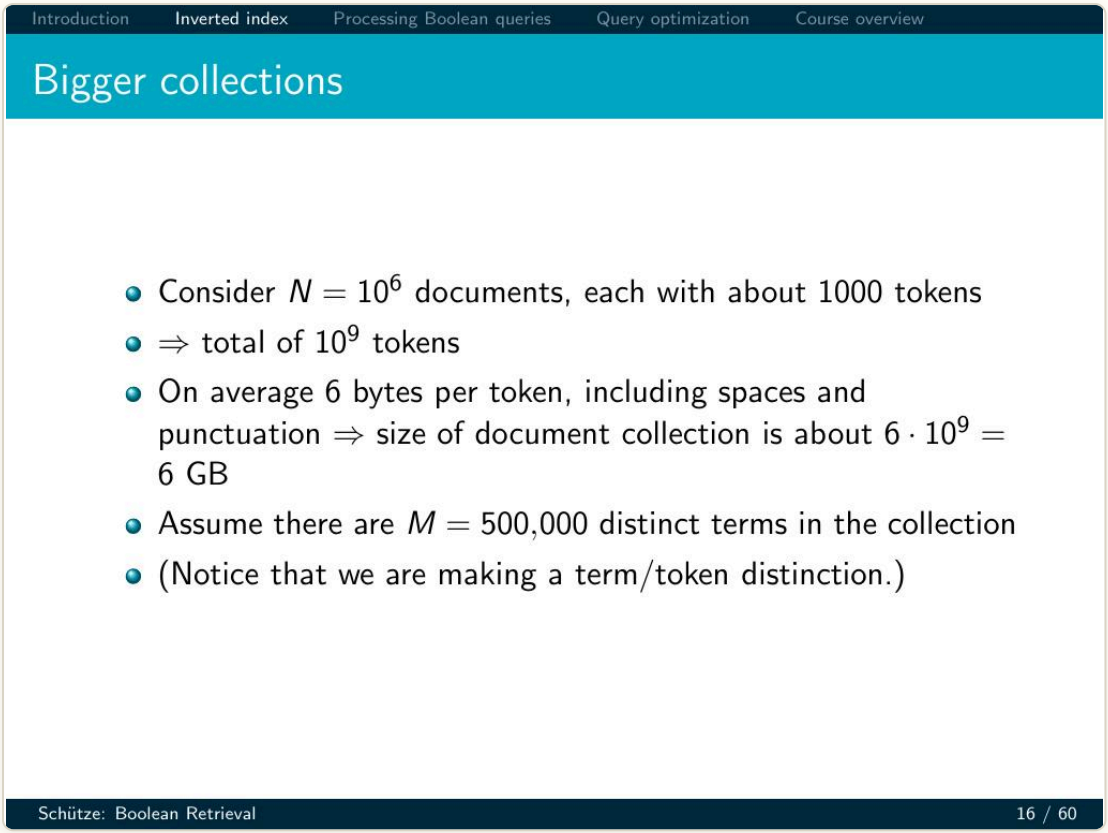


질의의 답

- **Anthony and Cleopatra** 3막 2장 — 아그리파: 안토니우스는 줄리어스 시저가 죽은 걸 보고 울부짖다시피 울었고, 필리피에서 브루투스가 죽은 걸 보고도 울었다
- **Hamlet** 3막 2장 — 폴로니어스: 나는 줄리어스 시저 역을 했지, 의사당에서 죽었어. 브루투스가 나를 죽였지

✍️ 메모

— 넘어가도 되는 쪽

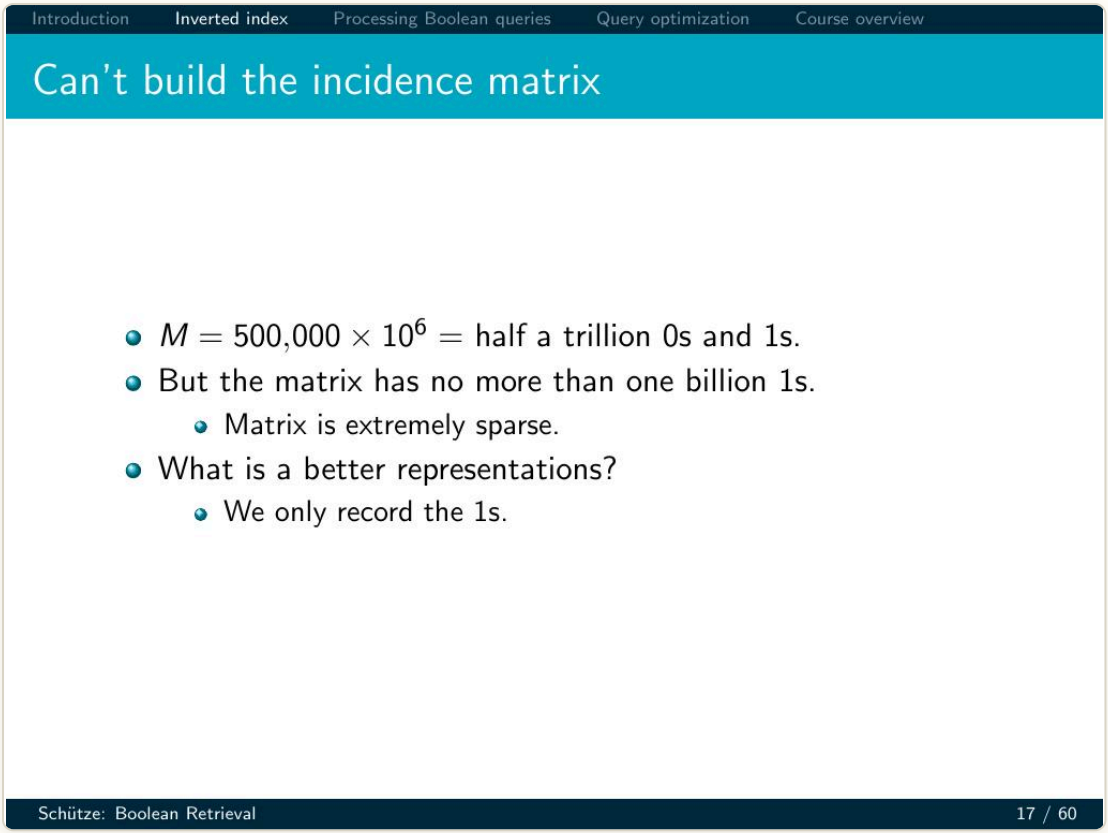


더 큰 컬렉션

- 문서 **N = 10^6**개, 문서마다 토큰 약 **1000**개
  - ⇒ 전체 **10^9** 토큰
- 토큰당 평균 6바이트(공백·문장부호 포함) ⇒ 컬렉션 크기 약 **$6 \cdot 10^9$ = 6GB**
- 서로 다른 단어(term)가 **M = 500,000**개라고 하자
  - (**term과 token을 구분**하고 있다는 점에 주의)

✍ 메모

★ 시험 token = 등장 하나하나 / term = 서로 다른 단어 종류

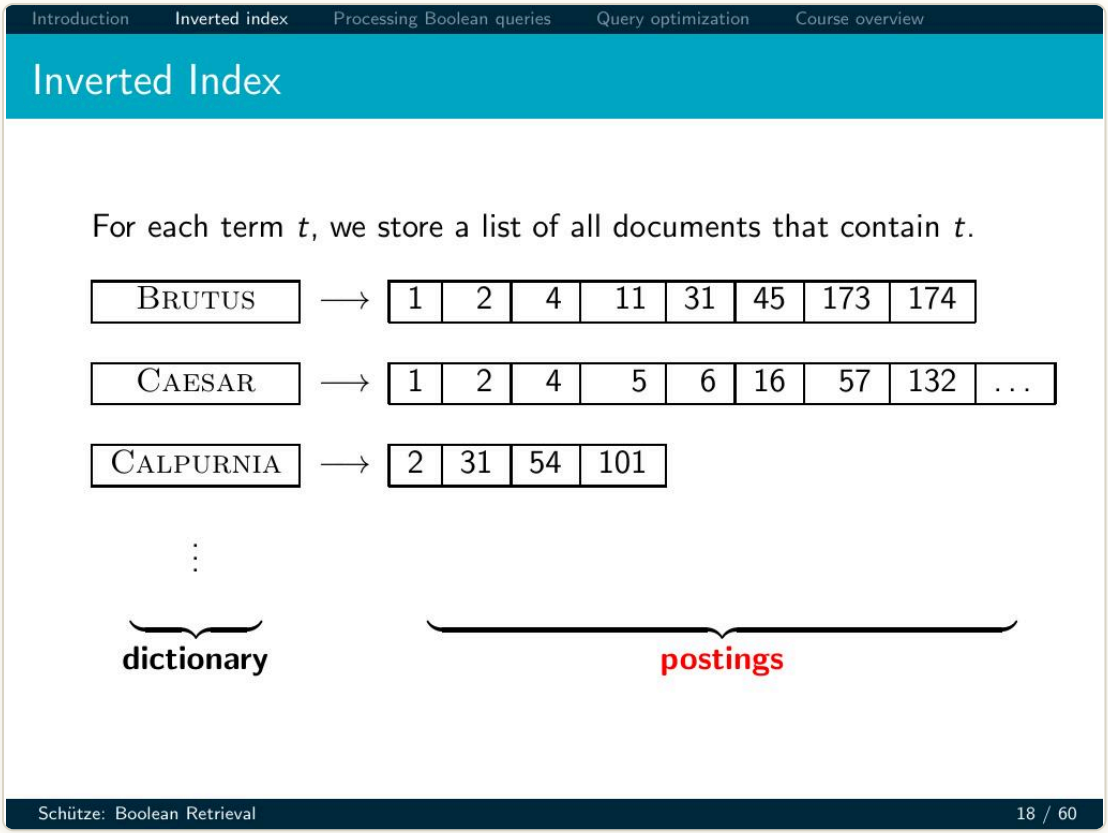


발생 행렬은 못 만든다

- M × N = 500,000 × 10^6 = **5천억 개**의 0과 1
- 하지만 1은 **10억 개를 넘지 않는다** (토큰이 10^9개뿐이니까)
- 행렬이 **극도로 희소(sparse)**하다
- 더 나은 표현은? → **1만 기록**하자

✍️ 메모

★ 시험 희소성이 역색인을 쓰는 이유

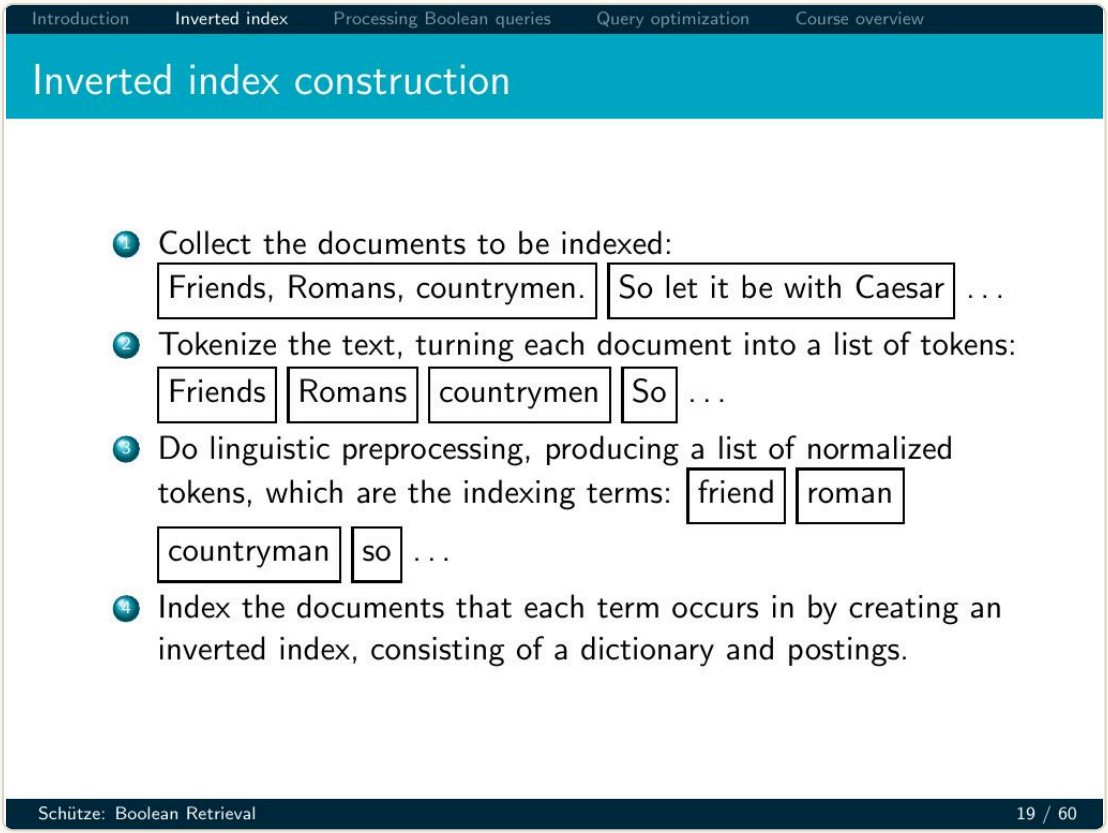


역색인 (Inverted index)

- 단어 t마다, t를 포함한 **모든 문서의 목록**을 저장
  - Brutus → 1, 2, 4, 11, 31, 45, 173, 174
  - Caesar → 1, 2, 4, 5, 6, 16, 57, 132, ...
  - Calpurnia → 2, 31, 54, 101
- 왼쪽 = **사전(dictionary)**, 오른쪽 = **포스팅(postings)**

✍ 메모

★ 시험 역색인 = 사전 + 포스팅 리스트, 포스팅은 docID 오름차순

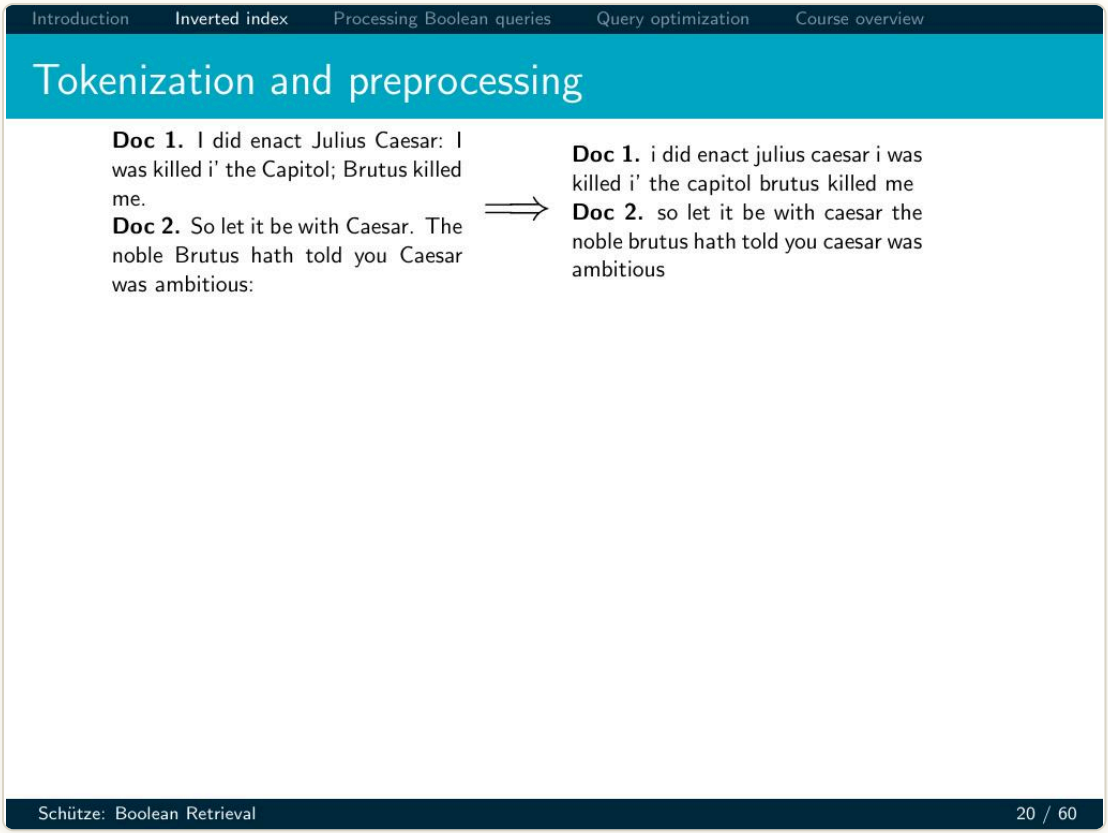


역색인 구축 과정

- ① 색인할 문서 모으기: Friends, Romans, countrymen. So let it be with Caesar ...
- ② 텍스트 **토큰화**: Friends | Romans | countrymen | So | ...
- ③ **언어 전처리** → 색인 단어(term)가 될 토큰: friend roman countryman so ...
- ④ 문서를 색인해 **사전과 포스팅**으로 된 역색인 만들기

✍️ 메모

★ 시험 구축 4단계: 수집 → 토큰화 → 정규화(전처리) → 색인

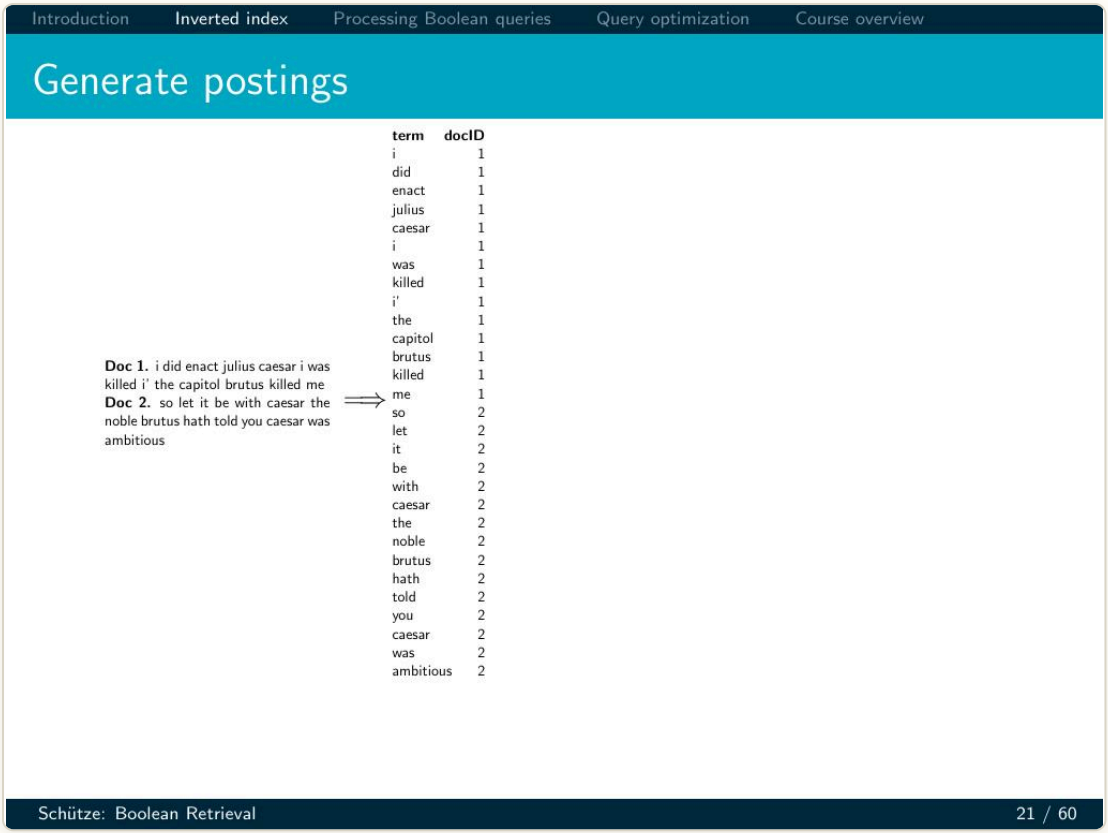


토큰화와 전처리

- Doc 1: I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me.
- Doc 2: So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious:
- ⇒ 소문자화·문장부호 제거: 'i did enact julius caesar i was killed i' the capitol brutus killed me' / 'so let it be with caesar the noble brutus hath told you caesar was ambitious'

✍️ 메모

— 넘어가도 되는 쪽

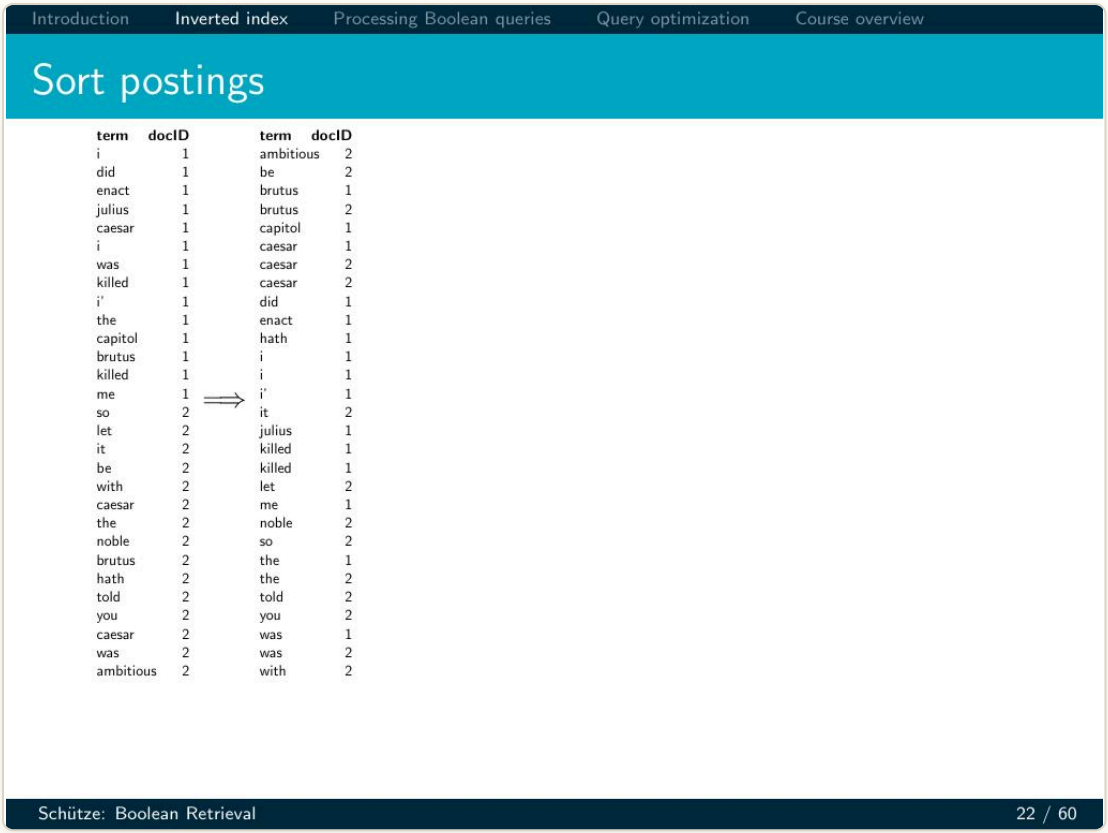


포스팅 생성

- 토큰마다 **(term, docID)** 쌍을 등장 순서대로 만든다
  - i 1, did 1, enact 1, julius 1, caesar 1, … so 2, let 2, … ambitious 2

✍️ 메모

— 넘어가도 되는 쪽

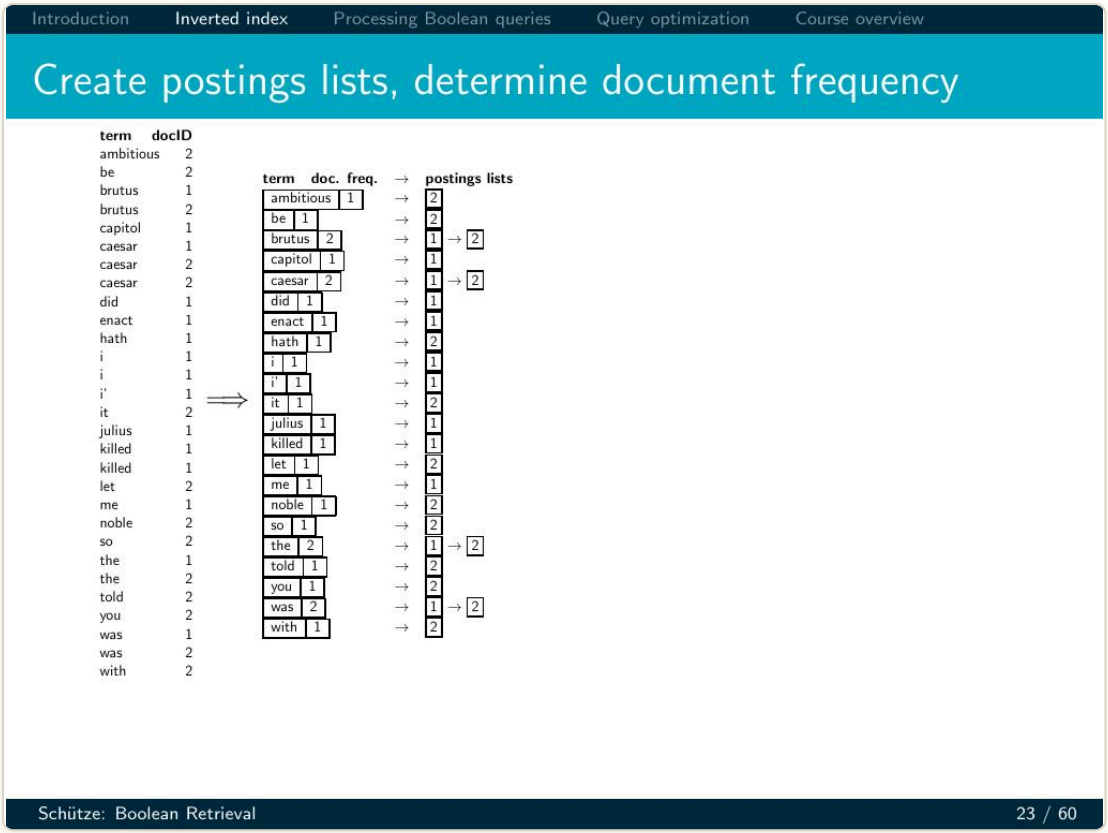


포스팅 정렬

- (term, docID) 쌍을 **term 알파벳순**, 같은 term이면 **docID순**으로 정렬
  - ambitious 2, be 2, brutus 1, brutus 2, capitol 1, caesar 1, caesar 2, caesar 2, ...

✍ 메모

★ 시험 정렬이 색인 구축의 핵심 단계 (4강 BSBI·SPIMI로 이어짐)

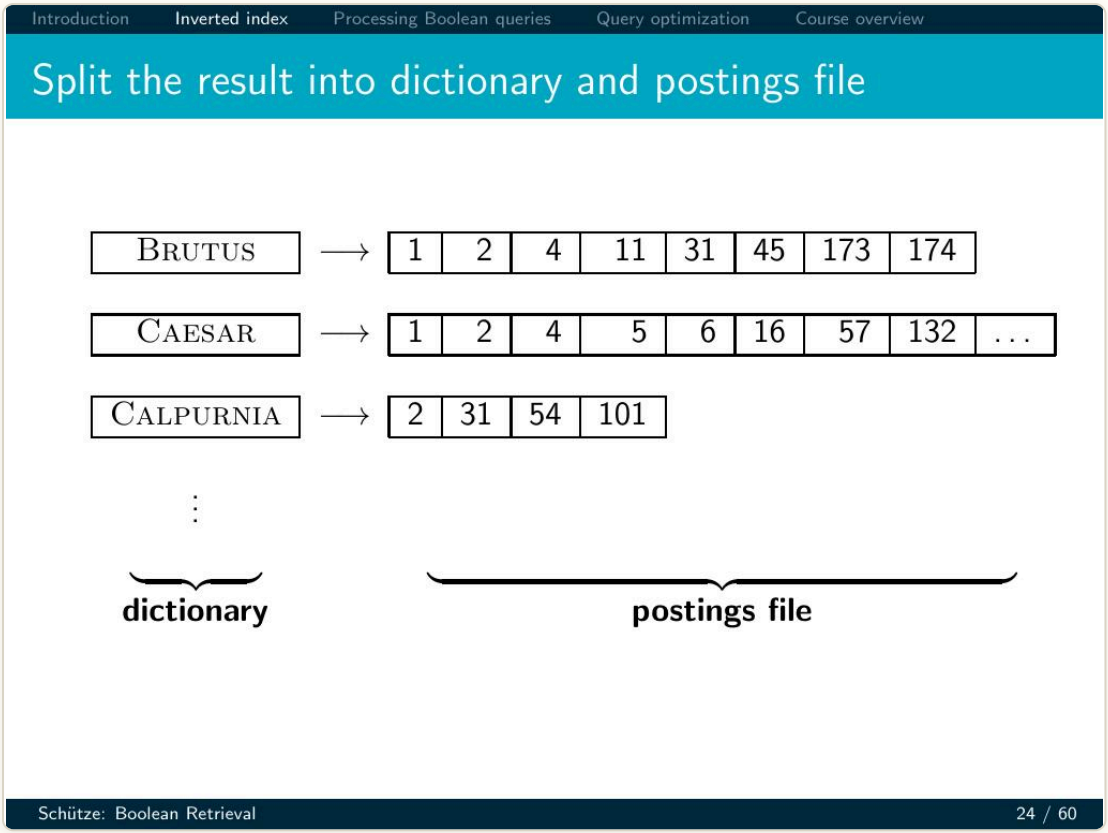


포스팅 리스트 만들기, 문서 빈도 구하기

- 같은 term의 항목을 합쳐 **포스팅 리스트**를 만들고, 몇 개 문서에 나오는지 **문서 빈도(df)**를 적는다
  - 예: brutus (df 2) → 1 → 2 / caesar (df 2) → 1 → 2 / ambitious (df 1) → 2
  - 같은 문서에 두 번 나와도(caesar in Doc 2) 포스팅엔 한 번

✍ 메모

★ 시험 **df = document frequency = 그 단어가 나오는 문서 수**

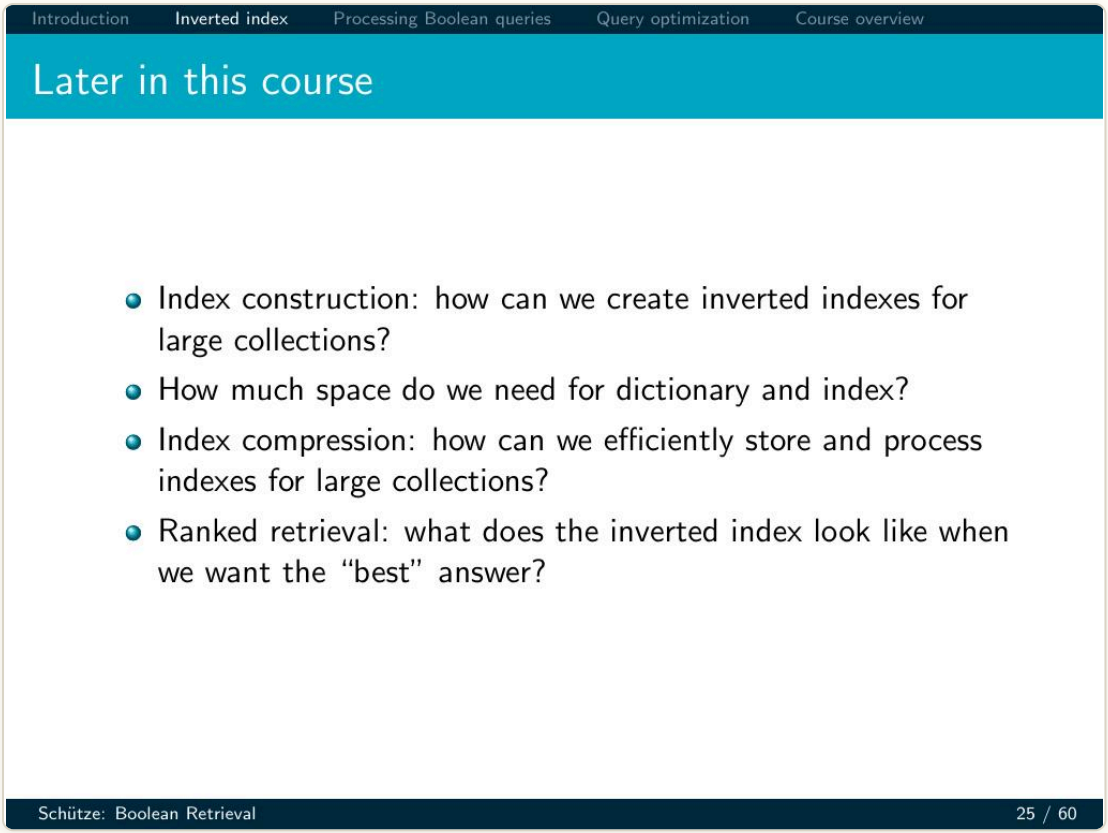


사전과 포스팅 파일로 나누기

- **사전(dictionary)** — 단어(와 df), 보통 메모리에
- **포스팅 파일(postings file)** — 각 단어의 docID 리스트, 보통 디스크에

✍️ 메모

— 넘어가도 되는 쪽

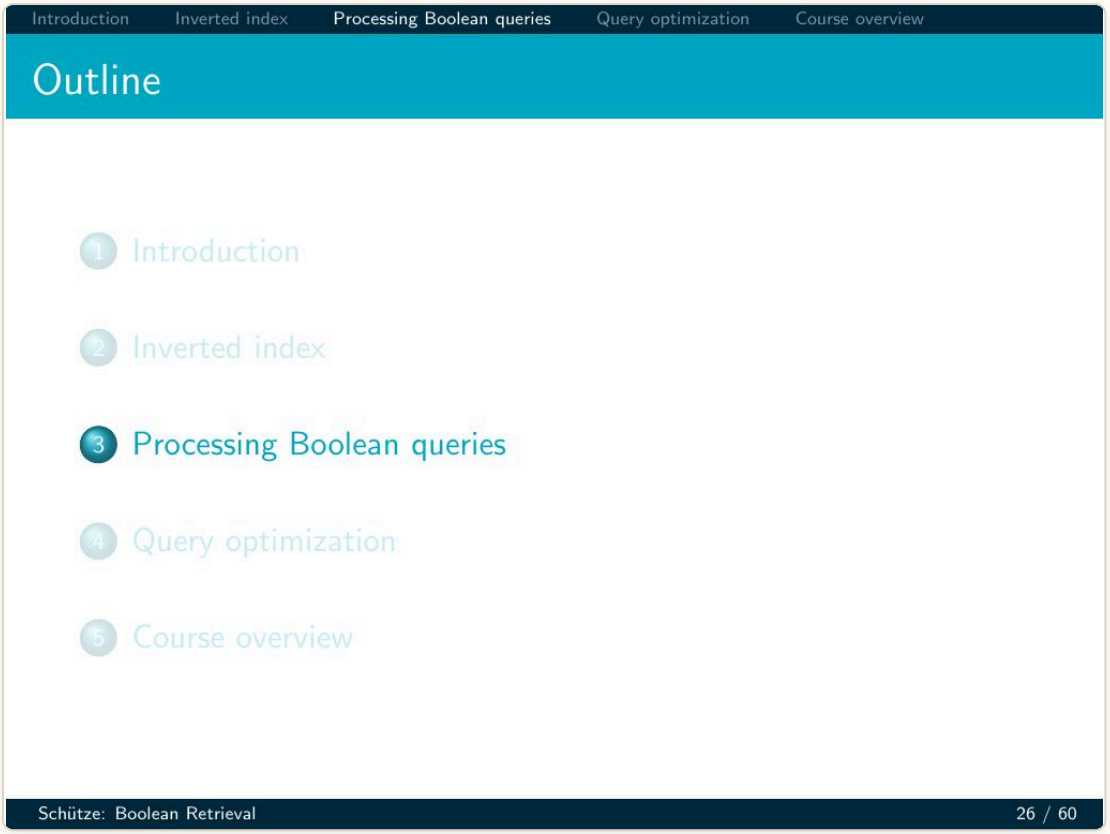


이 수업에서 나중에 다룰 것

- **색인 구축**: 큰 컬렉션의 역색인은 어떻게 만드나? (4강)
- 사전과 색인에 **공간**이 얼마나 필요한가?
- **색인 압축**: 큰 컬렉션의 색인을 어떻게 효율적으로 저장·처리하나? (5강)
- **순위 검색**: '가장 좋은' 답을 원할 때 역색인은 어떤 모습인가? (6·7강)

✍️ 메모

— 넘어가도 되는 쪽

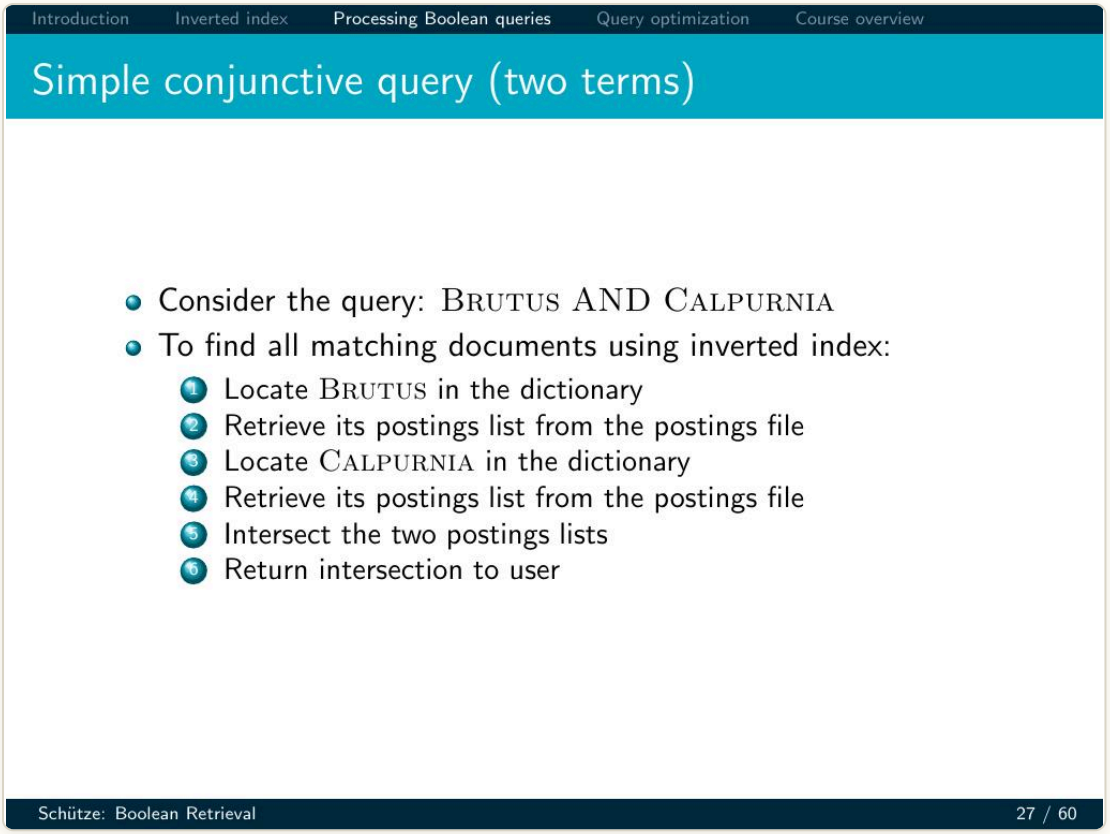


목차

- 소개 → 역색인 → 불리언 질의 처리 → 질의 최적화 → 과정 개관
- 지금: **불리언 질의 처리**

✍️ 메모

— 넘어가도 되는 쪽

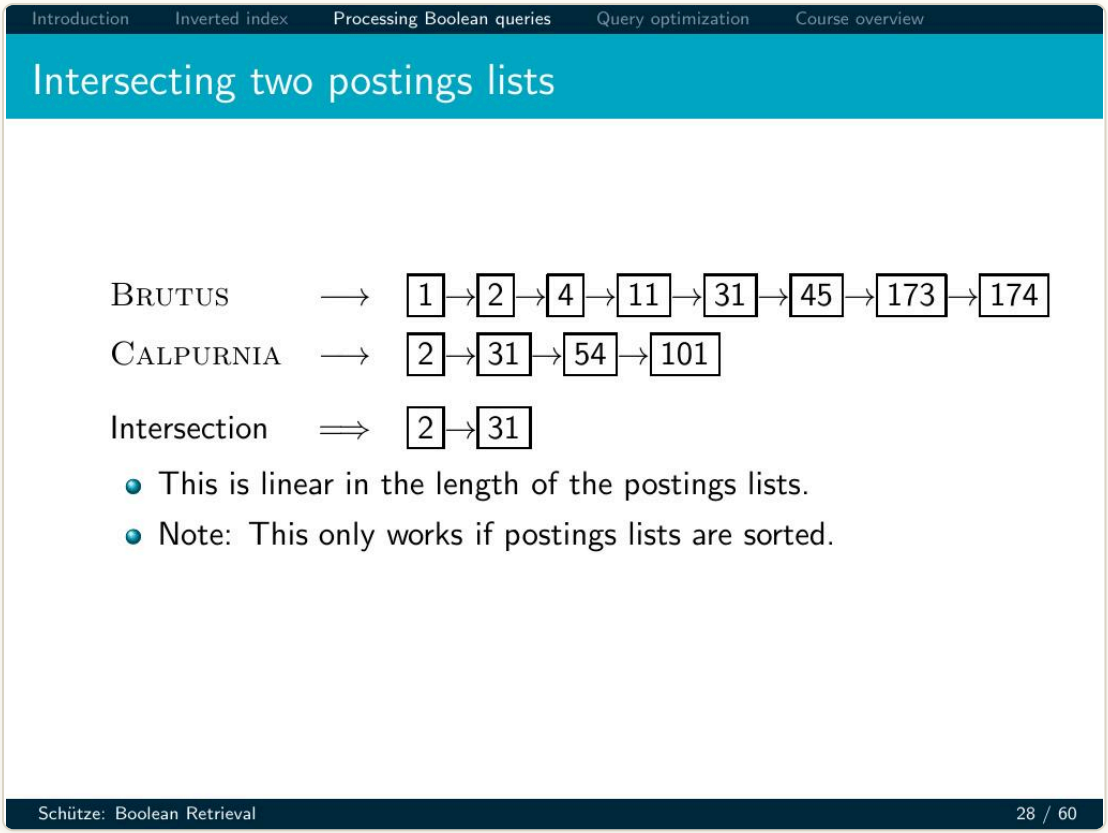


간단한 AND 질의 (단어 2개)

- 질의: **Brutus AND Calpurnia**
- 역색인으로 맞는 문서 찾기
  - ① 사전에서 Brutus 찾기
  - ② 포스팅 파일에서 그 포스팅 리스트 가져오기
  - ③ 사전에서 Calpurnia 찾기
  - ④ 그 포스팅 리스트 가져오기
  - ⑤ 두 포스팅 리스트의 **교집합**
  - ⑥ 교집합을 사용자에게 반환

✍️ 메모

— 넘어가도 되는 쪽

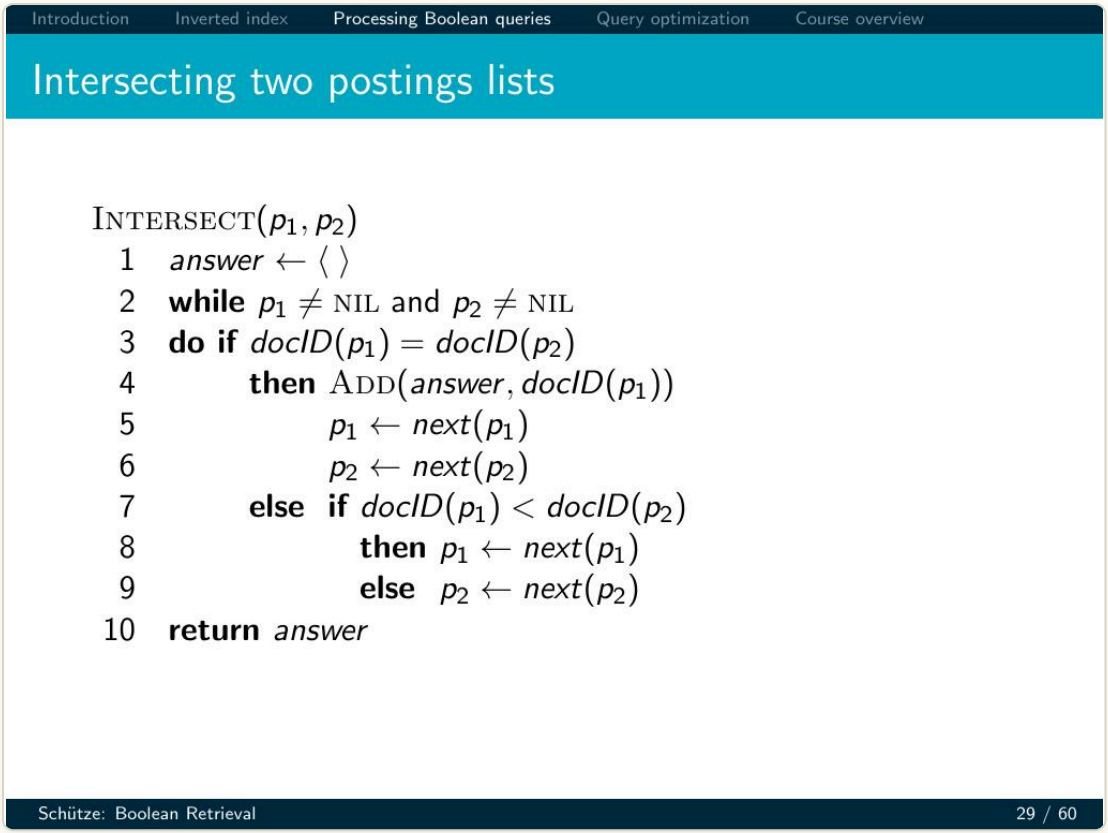


두 포스팅 리스트의 교집합

- Brutus → 1 2 4 11 31 45 173 174
- Calpurnia → 2 31 54 101
- 교집합 ⇒ **2, 31**
- 포스팅 리스트 길이에 대해 **선형 시간** — O(x + y)
- 주의: 포스팅 리스트가 **정렬돼 있어야만** 된다

✍ 메모

★ 시험 merge(교집합) = 두 포인터를 앞으로 걸으며 비교, O(x+y), 정렬 필수

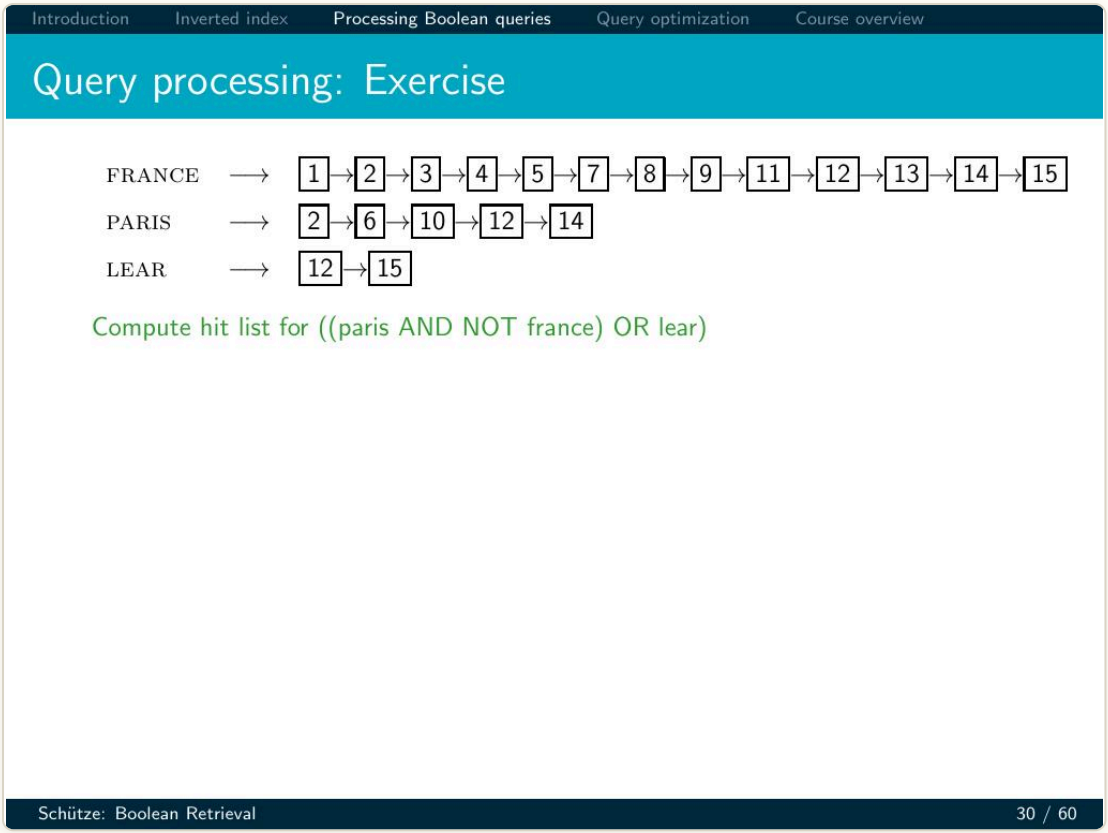


교집합 알고리즘 INTERSECT(p1, p2)

```
answer ← ⟨ ⟩
while p1 ≠ NIL and p2 ≠ NIL
  if docID(p1) = docID(p2): ADD(answer, docID(p1)); p1 ← next(p1); p2 ← next(p2)
  else if docID(p1) < docID(p2): p1 ← next(p1)
  else: p2 ← next(p2)
return answer
```

✍ 메모

★ 시험 같으면 둘 다 전진, 다르면 작은 쪽만 전진 — 손으로 따라가기 문제 대비



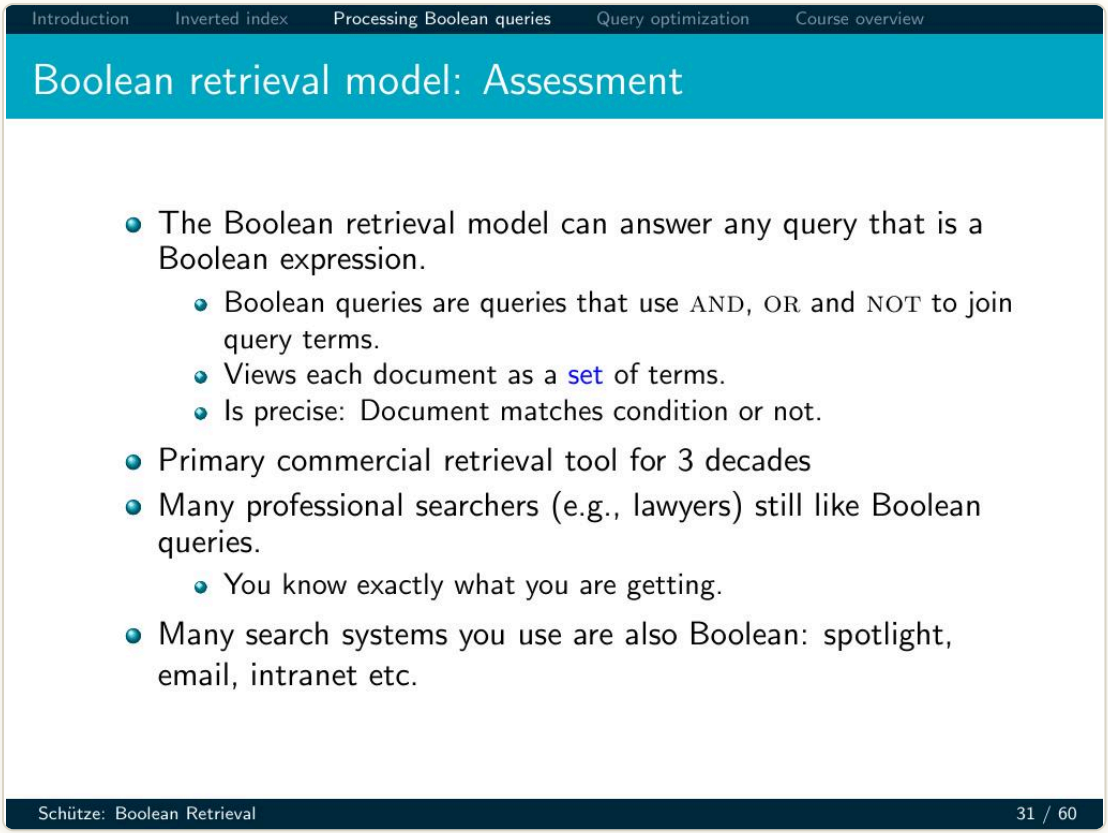
질의 처리 연습

- France → 1 2 3 4 5 7 8 9 11 12 13 14 15
- Paris → 2 6 10 12 14
- Lear → 12 15
- ((paris AND NOT france) OR lear)의 결과 문서는?

✍️ 메모

✏ 예·풀이 paris AND NOT france = {6, 10} → OR lear {12, 15} → 답 {6, 10, 12, 15}

💡 이해 AND NOT도 merge로 선형 시간에 가능: paris에서 france에 있는 것만 건너뛴다

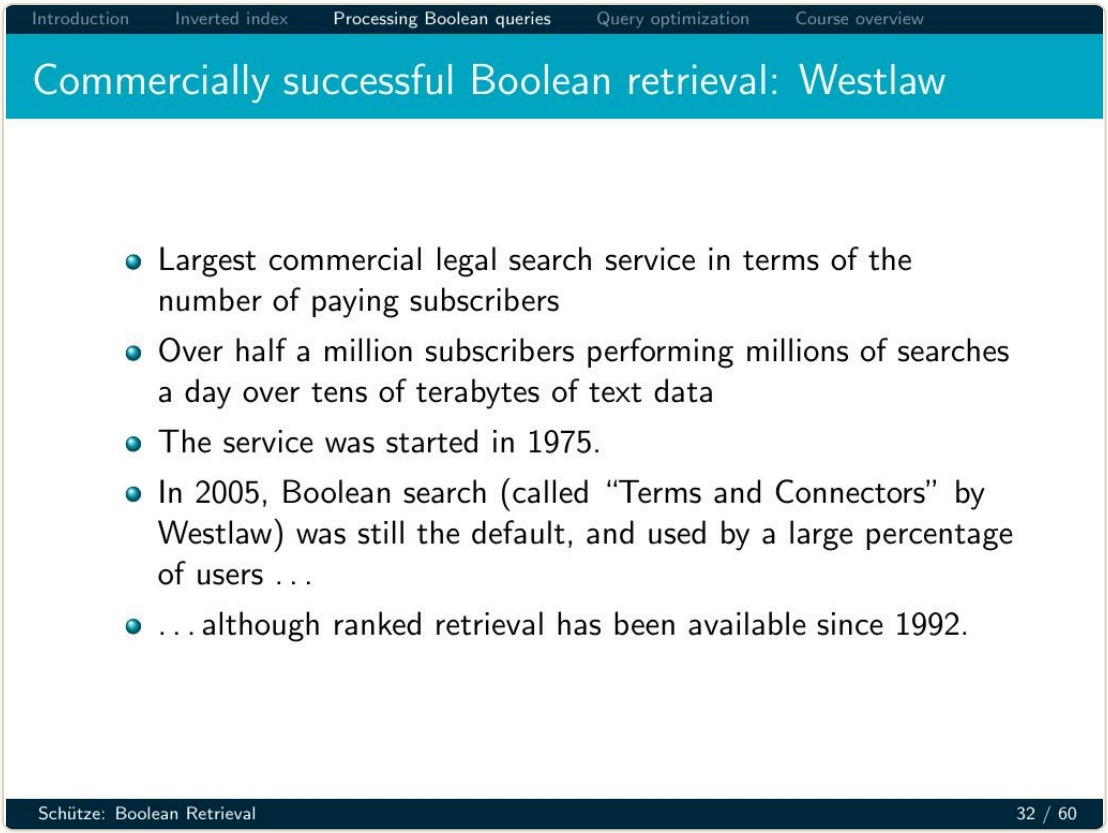


불리언 검색 모델: 평가

- 불리언 식이면 **어떤 질의든** 답할 수 있다
  - 불리언 질의 = and, or, not으로 질의어를 잇는 것
- 문서를 **단어의 집합**으로 본다
- **정확(precise)**: 문서가 조건을 만족하거나 안 하거나
- 30년간 **주된 상용 검색 도구**
- 많은 전문 검색자(예: **변호사**)는 아직 불리언 질의를 좋아한다
  - 무엇을 얻는지 **정확히 안다**
- 많은 검색 시스템이 지금도 불리언: 스포트라이트, 이메일, 인트라넷 등

✍ 메모

★ 시험 불리언 = 문서를 단어 '집합'으로 봄 (빈도·순서 무시)

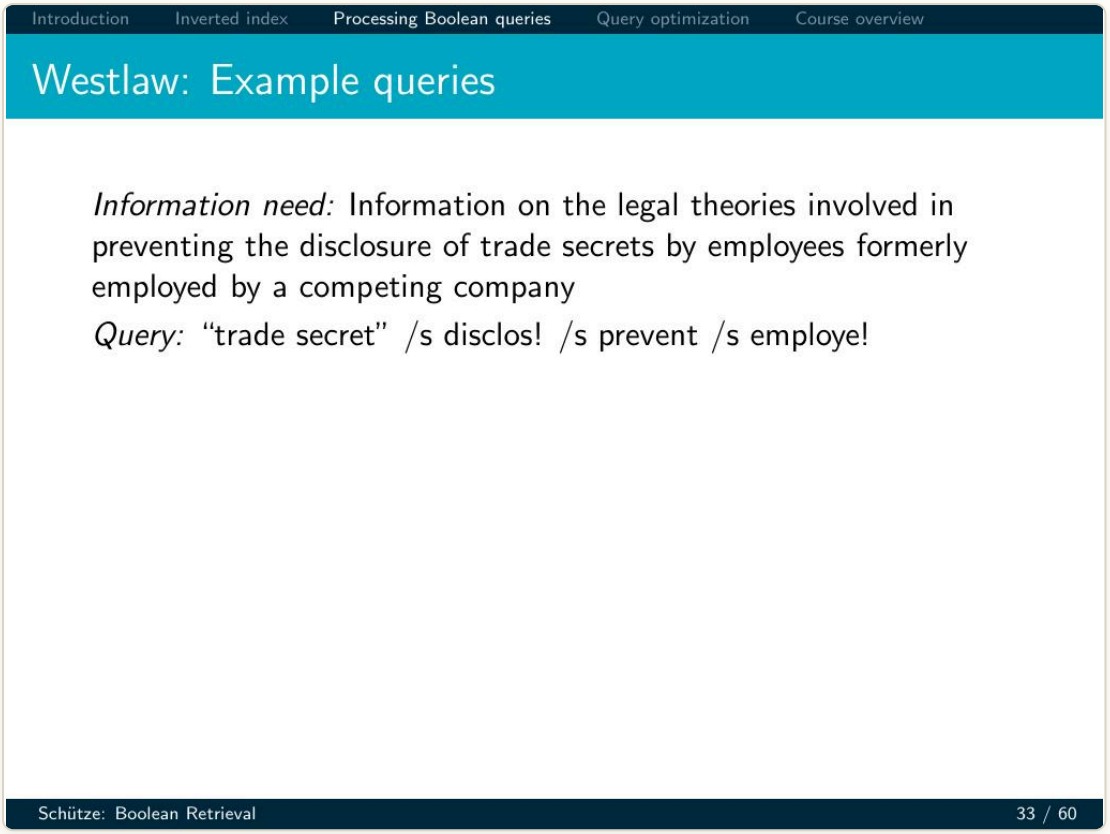


상업적으로 성공한 불리언 검색: Westlaw

- 유료 구독자 수 기준 **최대의 상용 법률 검색 서비스**
- 50만 명 넘는 구독자가 수십 TB 텍스트에 하루 수백만 건 검색
- **1975년** 시작
- 2005년에도 불리언 검색(Westlaw는 'Terms and Connectors'라 부름)이 **기본값**이었고 많은 사용자가 썼다
  - 순위 검색은 **1992년**부터 있었는데도

✍ 메모

— 넘어가도 되는 쪽

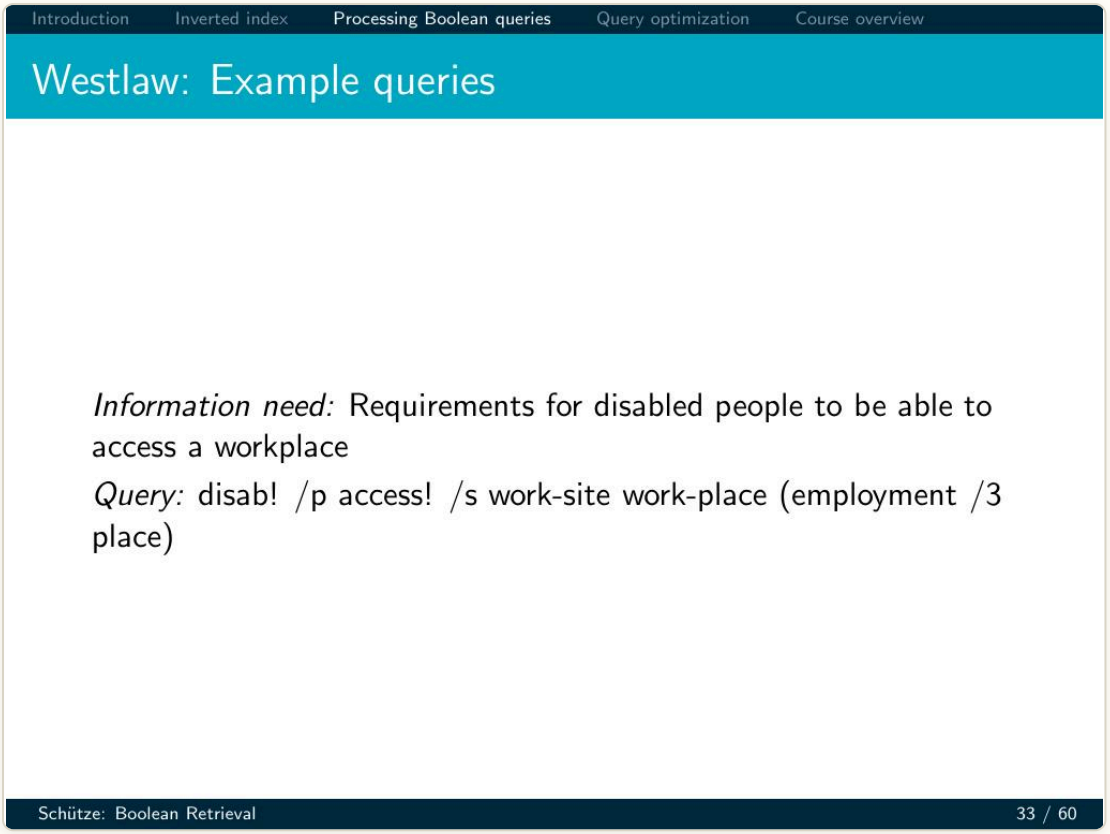


Westlaw 질의 예 ①

- 정보 요구: 경쟁사 출신 직원의 **영업비밀 공개를 막는** 법리
- 질의: $ "trade secret" /s disclos! /s prevent /s employe!

✍️ 메모

💡 이해 ! = 접미어 와일드카드 (disclos! → disclose, disclosure ...)

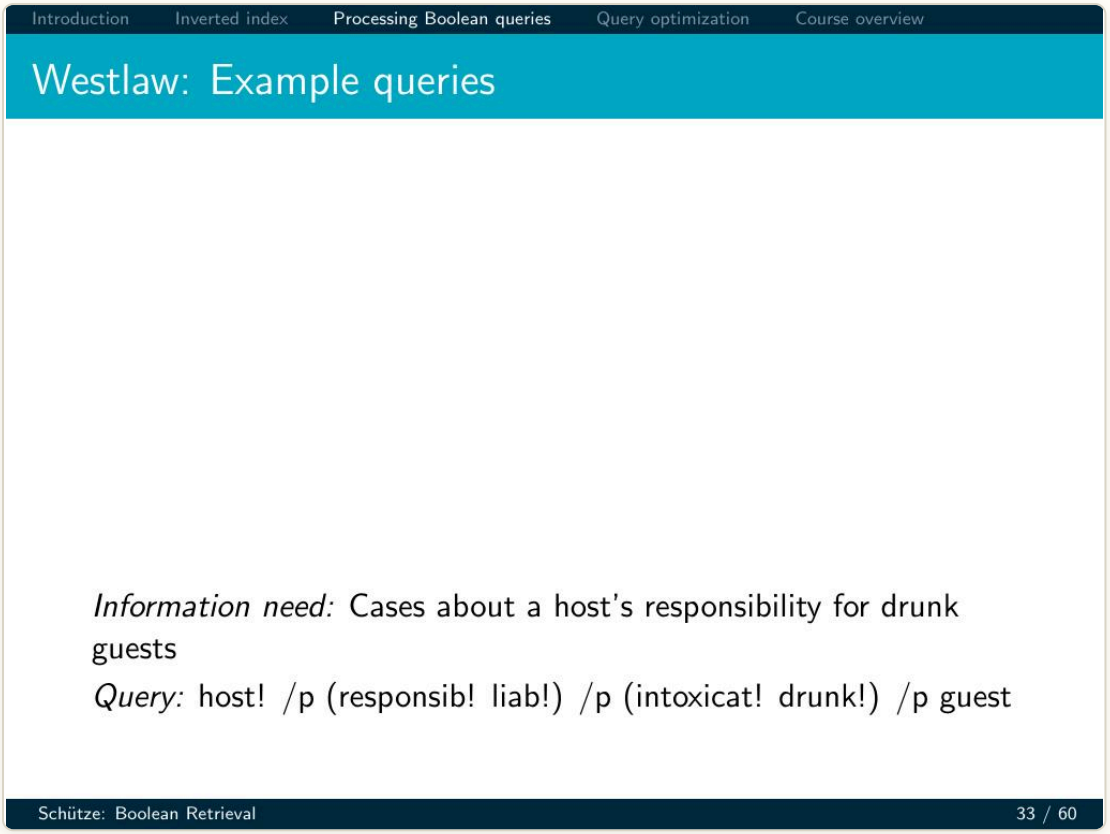


Westlaw 질의 예 ②

- 정보 요구: 장애인이 **작업장에 접근**할 수 있어야 한다는 요건
- 질의: $ disab! /p access! /s work-site work-place (employment /3 place)

✍ 메모

— 넘어가도 되는 쪽

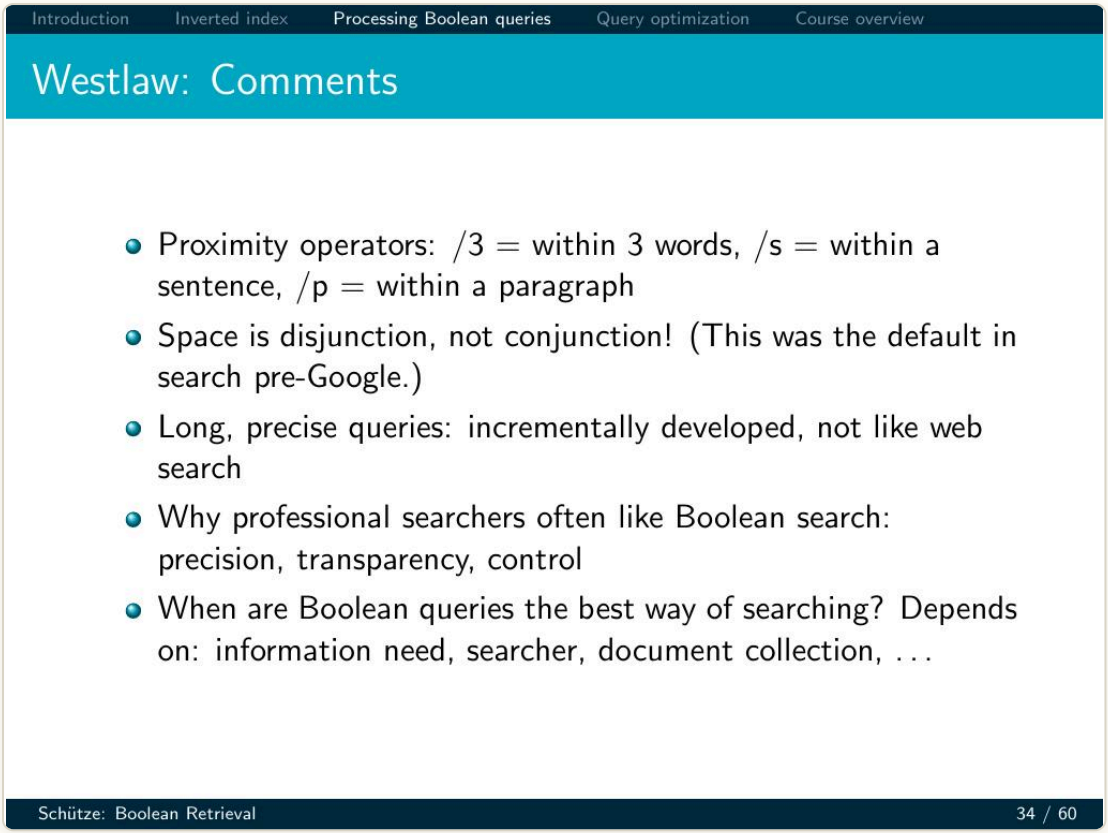


Westlaw 질의 예 ③

- 정보 요구: **술 취한 손님**에 대한 주인의 책임에 관한 판례
- 질의: $ host! /p (responsib! liab!) /p (intoxicat! drunk!) /p guest

✍ 메모

— 넘어가도 되는 쪽

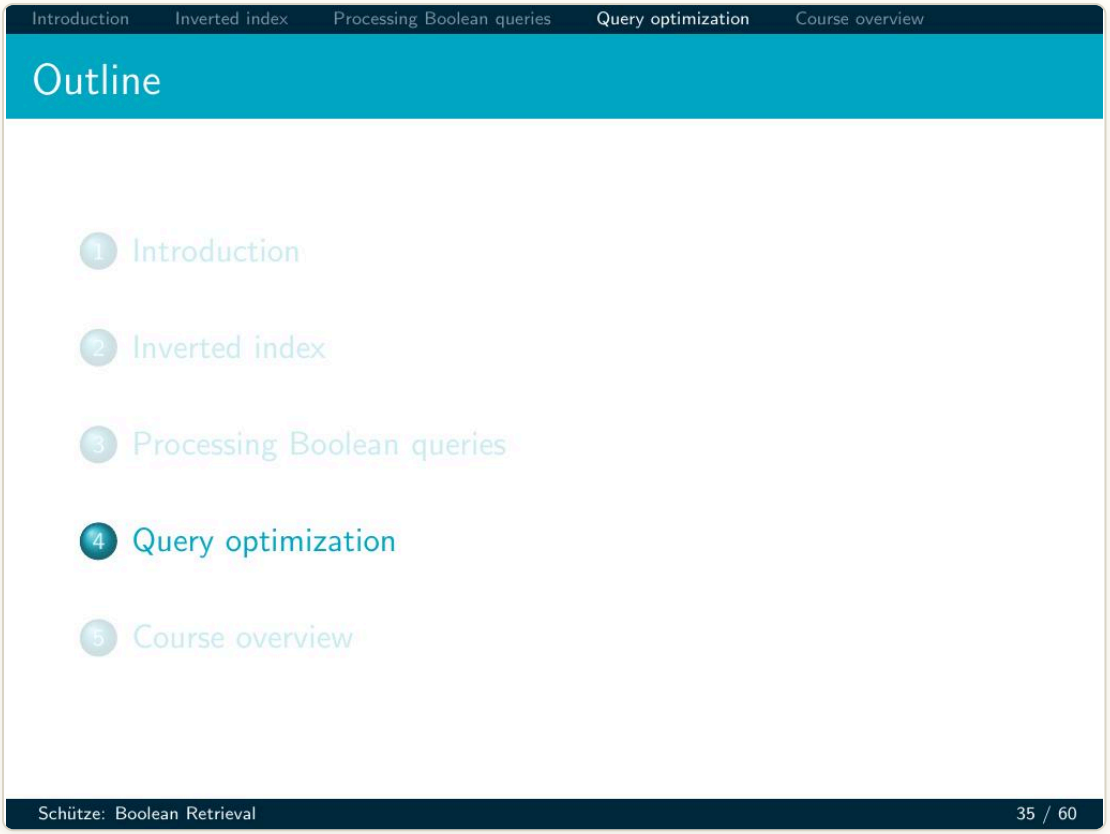


Westlaw: 정리

- **근접 연산자**: /3 = 3단어 이내, /s = 같은 문장, /p = 같은 문단
- **공백은 OR(disjunction)**이지 AND가 아니다! (구글 이전 검색의 기본값)
- 길고 정밀한 질의 — 웹 검색과 달리 **점진적으로** 다듬음
- 전문 검색자가 불리언을 좋아하는 이유: **정밀도, 투명성, 통제**
- 불리언이 최선인 경우는? 정보 요구, 검색자, 문서 컬렉션 ... 에 따라 다르다

✍ 메모

⚠ 주의 **Westlaw의 공백 = OR, 구글의 공백 = AND — 반대!**

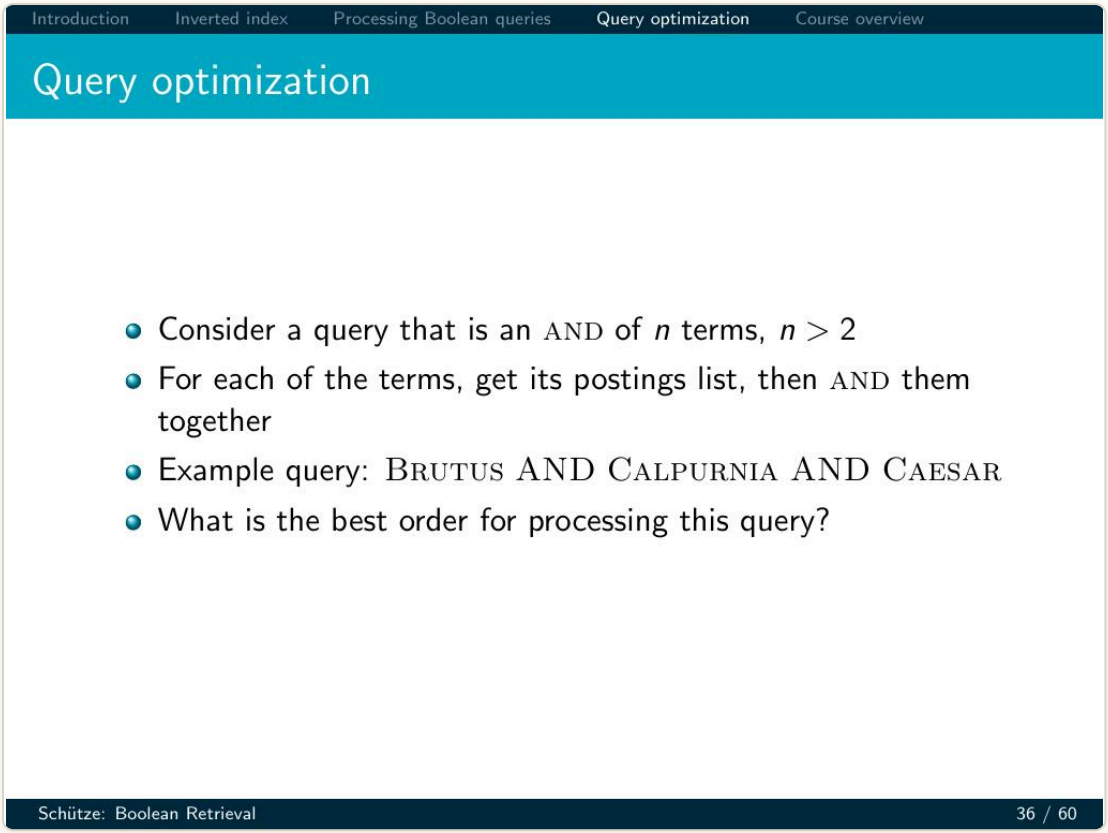


목차

- 소개 → 역색인 → 불리언 질의 처리 → 질의 최적화 → 과정 개관
- 지금: **질의 최적화**

✍️ 메모

— 넘어가도 되는 쪽

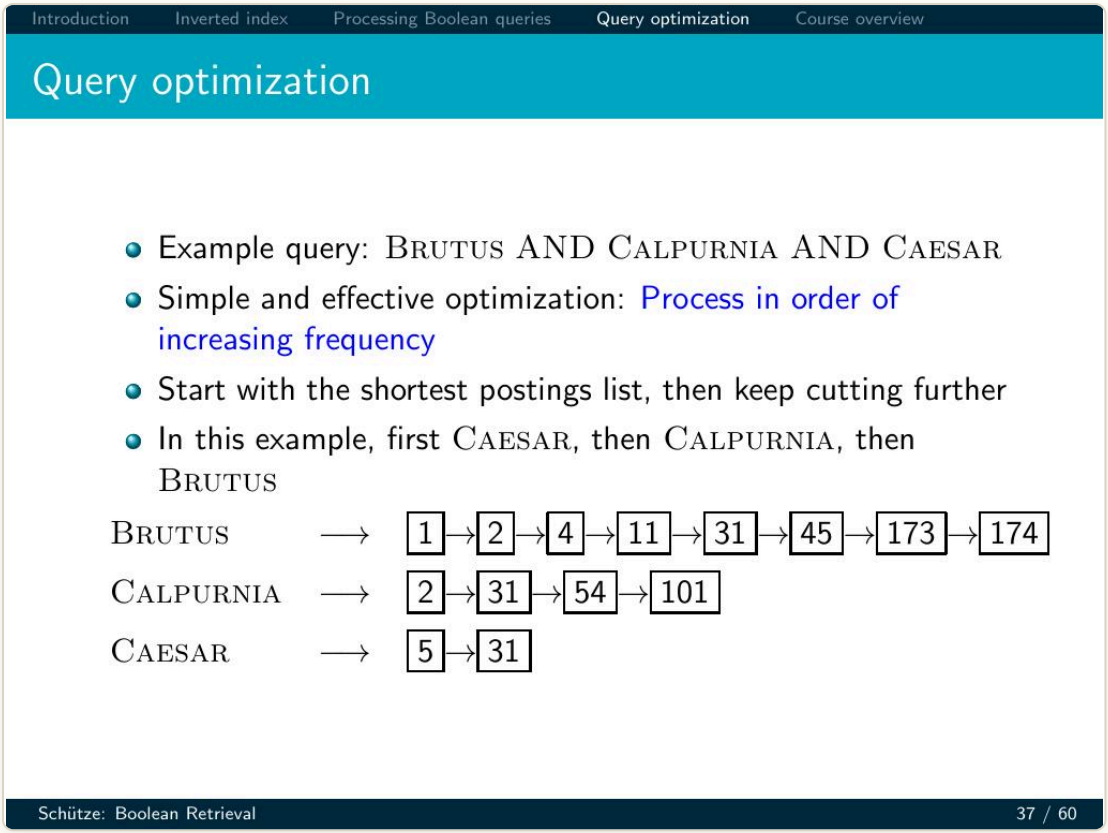


질의 최적화

- n > 2개 단어의 AND 질의
- 단어마다 포스팅 리스트를 가져와 AND
- 예: Brutus AND Calpurnia AND Caesar
- 처리 순서는 어떻게 하는 게 가장 좋을까?

✍ 메모

— 넘어가도 되는 쪽



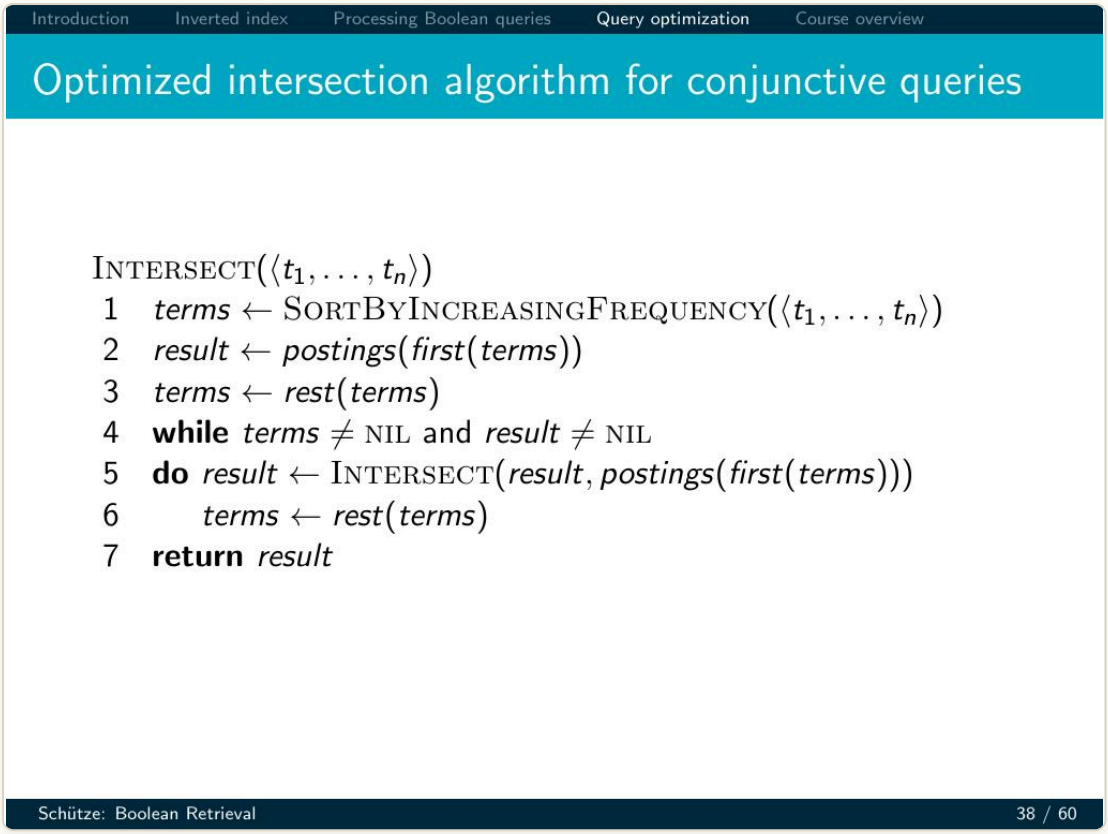
질의 최적화 — 빈도 오름차순

- 간단하고 효과적인 최적화: **빈도(df)가 작은 순서로** 처리
- **가장 짧은 포스팅 리스트부터** 시작해 계속 잘라 나간다
- 이 예에선 먼저 Caesar, 그다음 Calpurnia, 그다음 Brutus

✍ 메모

★ 시험 AND 최적화 = df 오름차순 (중간 결과가 절대 가장 짧은 리스트보다 커지지 않음)

⚠ 주의 이 슬라이드의 df 그림이 없어 순서만 나옴 — p.18 예시 길이로는 Calpurnia(4)가 가장 짧다. 시험엔 주어진 df로 판단

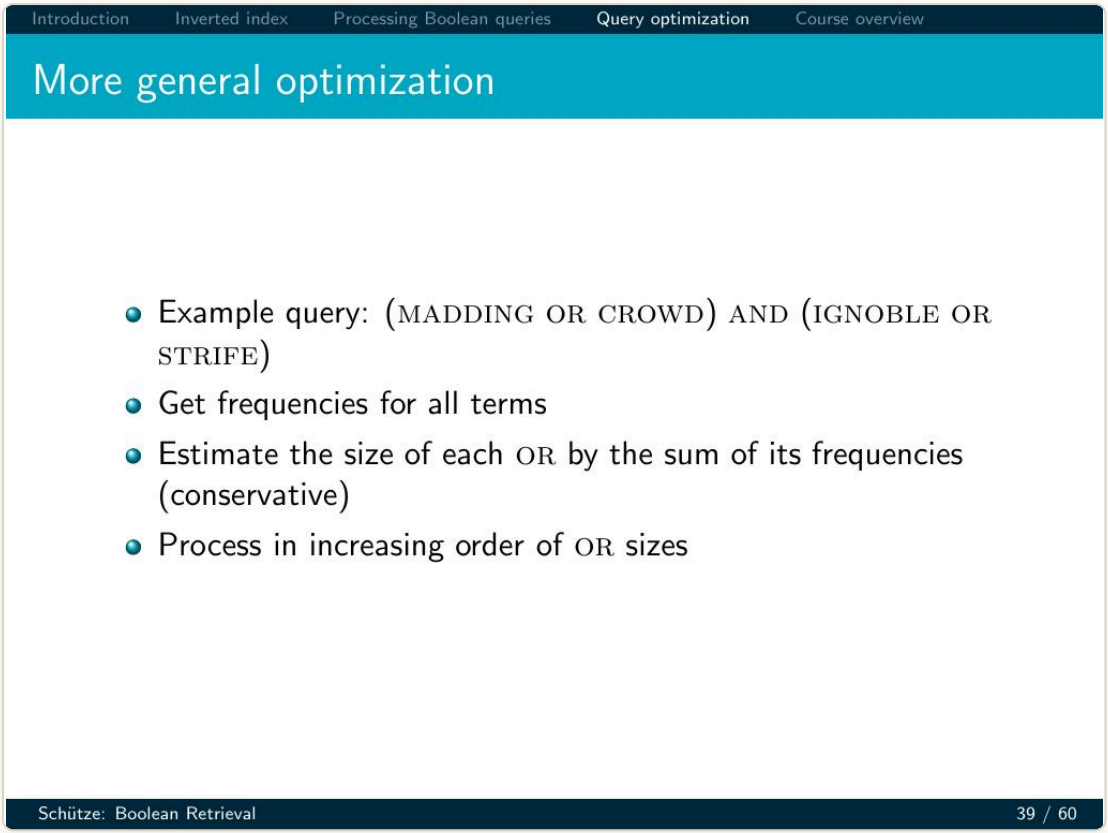


AND 질의용 최적화 교집합 알고리즘

```
INTERSECT(⟨t1, …, tn⟩)
terms ← SortByIncreasingFrequency(⟨t1, …, tn⟩)
result ← postings(first(terms))
terms ← rest(terms)
while terms ≠ NIL and result ≠ NIL
  result ← INTERSECT(result, postings(first(terms)))
  terms ← rest(terms)
return result
```

✍️ 메모

💡 이해 result가 비면 바로 멈춘다 (더 볼 필요 없음)

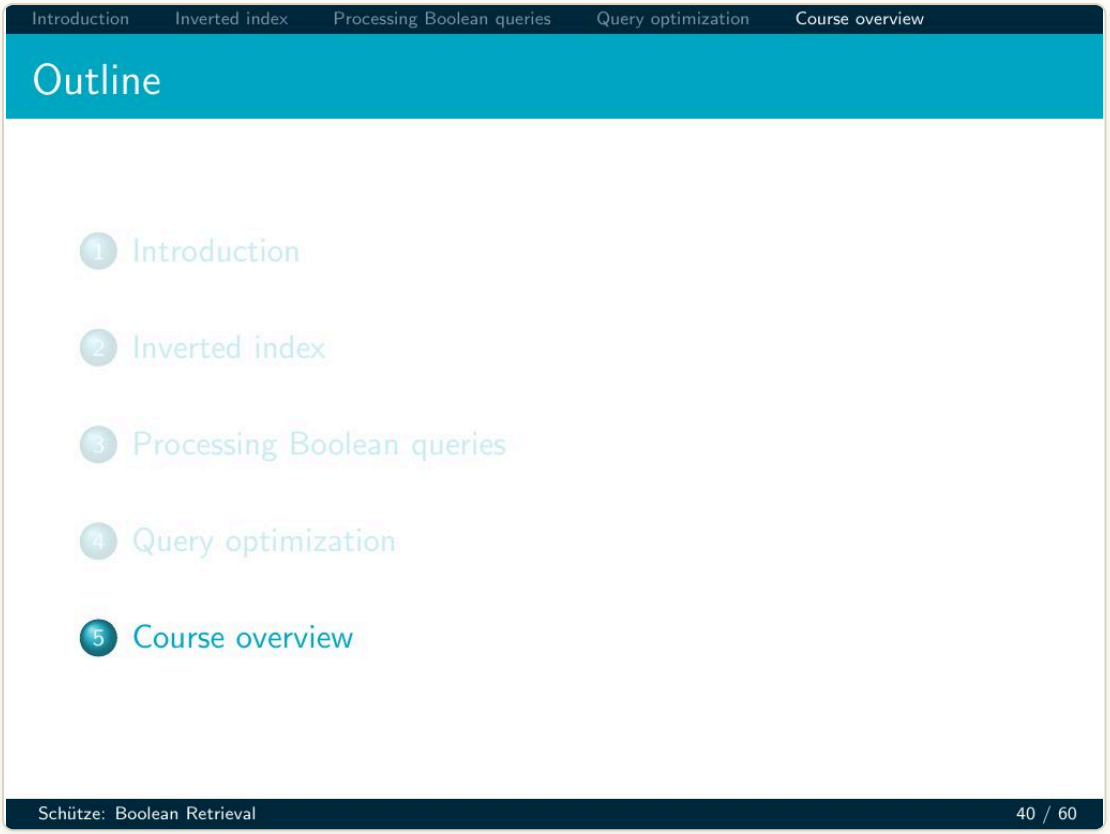


더 일반적인 최적화

- 예: (madding OR crowd) AND (ignoble OR strife)
- 모든 단어의 빈도를 구한다
- 각 OR의 크기를 빈도의 **합**으로 추정 (보수적 — 실제론 그 이하)
- OR 크기가 **작은 순서로** 처리

✍ 메모

★ 시험 OR 묶음 크기 ≤ df 합 → 합이 작은 묶음부터 AND

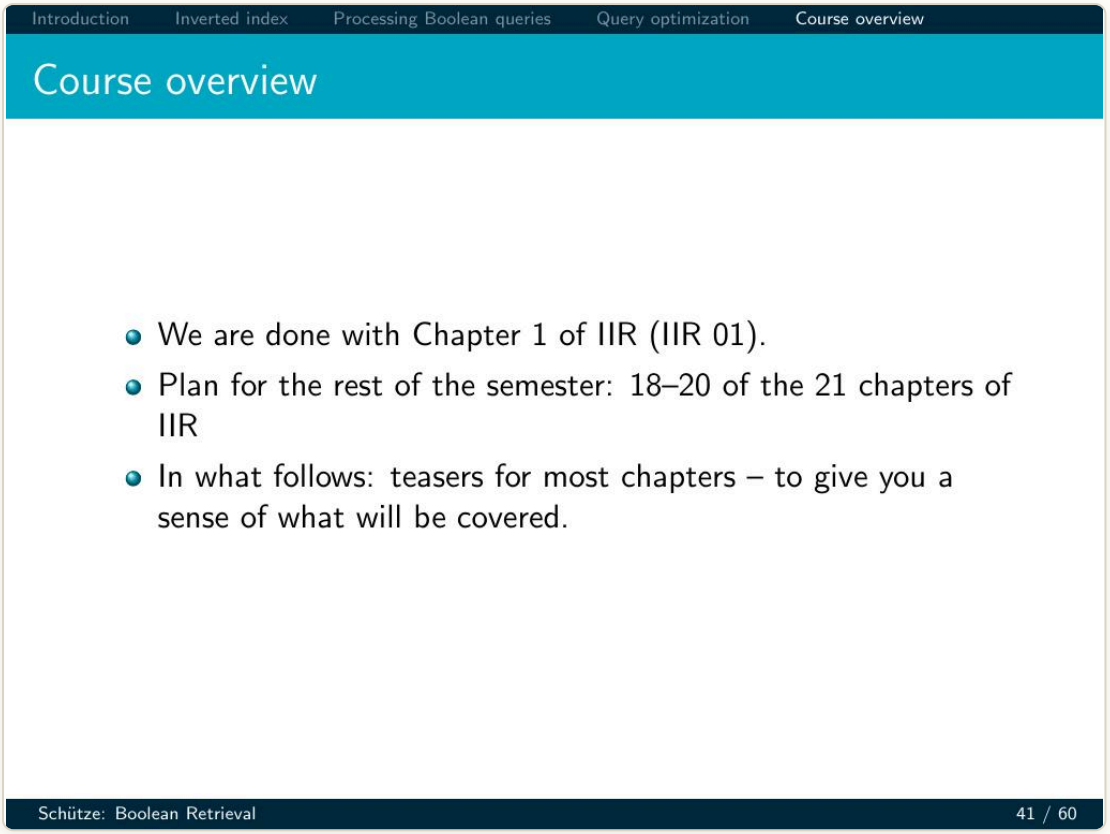


목차

- 소개 → 역색인 → 불리언 질의 처리 → 질의 최적화 → 과정 개관
- 지금: **과정 개관**

✍️ 메모

— 넘어가도 되는 쪽

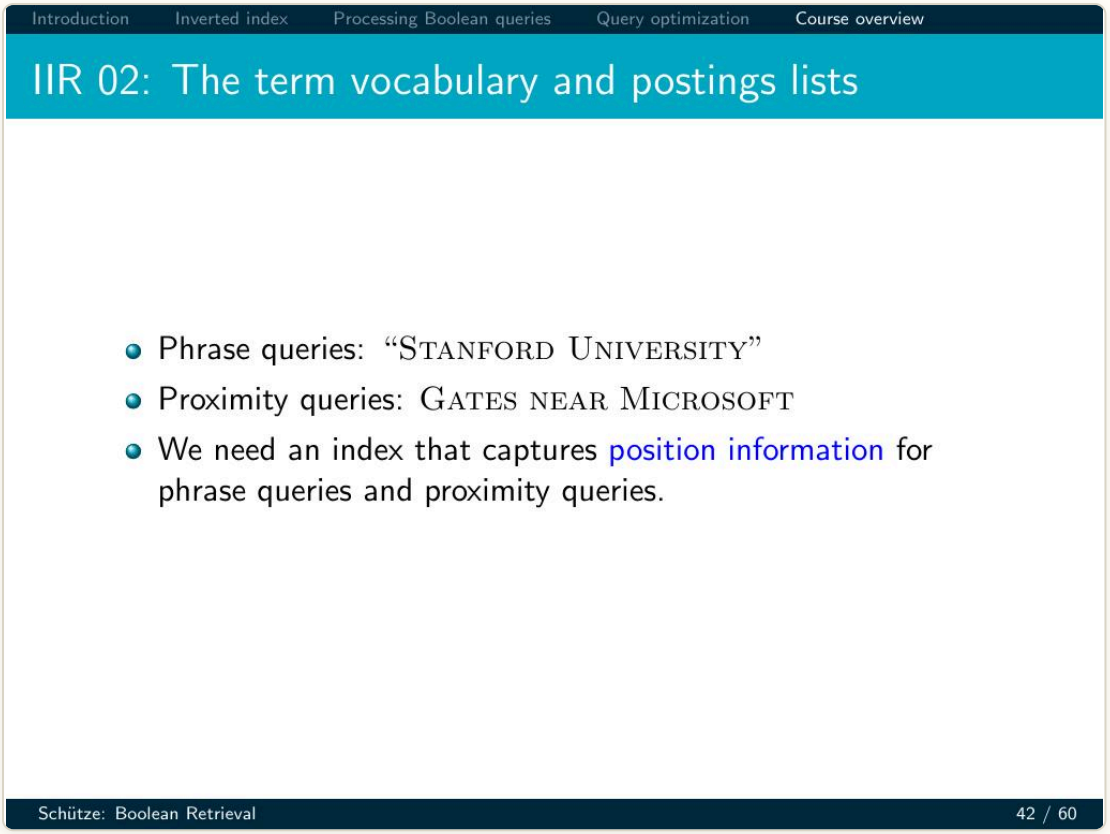


과정 개관

- IIR 1장을 끝냈다
- 남은 학기 계획: IIR 21장 중 18~20장
- 이어지는 쪽: 대부분 장의 맛보기

✍️ 메모

⚠ 주의 이 수업 중간고사 자료는 7강(IIR 7)까지 — 8장 이후 맛보기는 범위 밖일 가능성

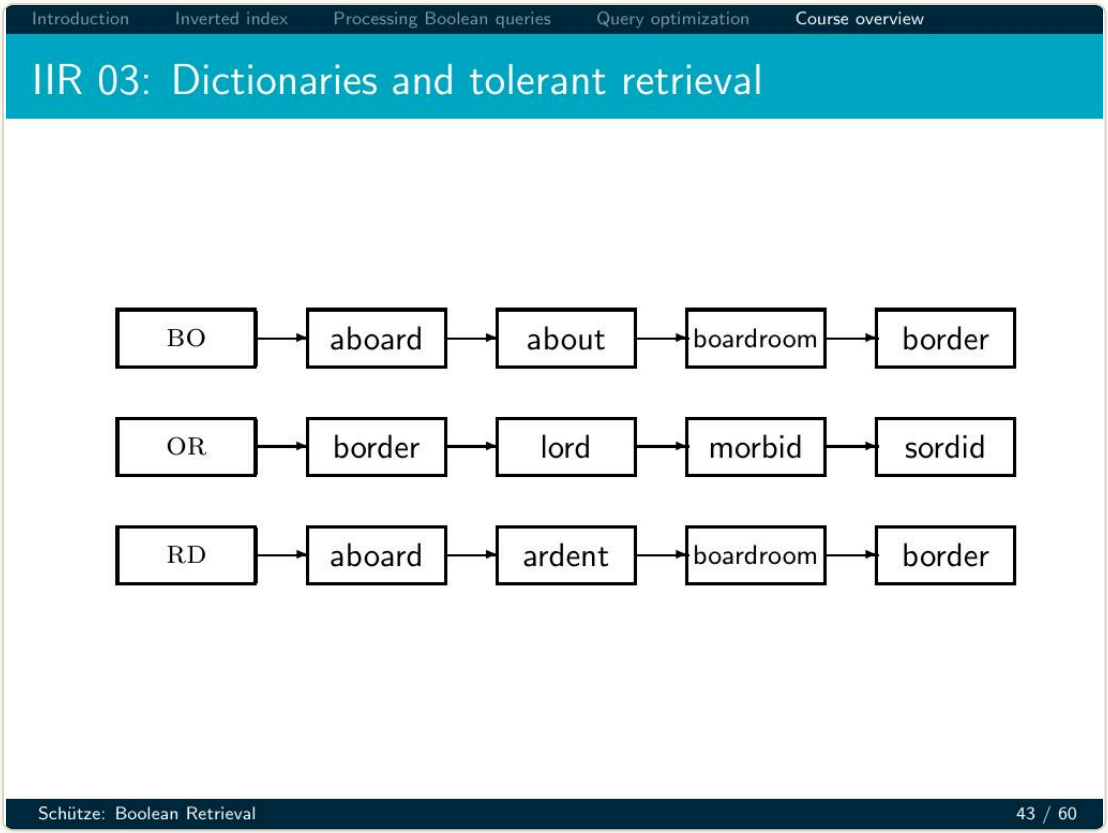


IIR 02: 용어 어휘와 포스팅 리스트

- **구 질의**: "Stanford University"
- **근접 질의**: Gates near Microsoft
- 구·근접 질의엔 **위치 정보**를 담은 색인이 필요

✍️ 메모

💡 이해 2강에서 위치 색인으로 다룸

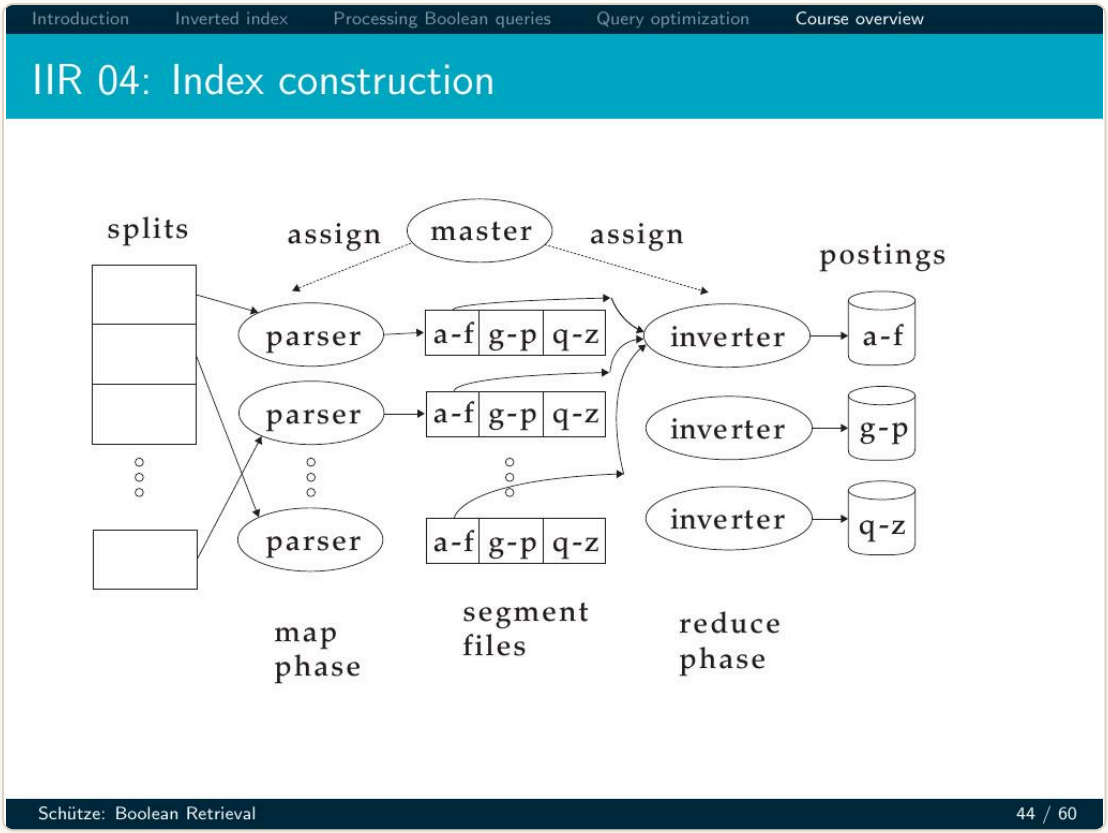


IIR 03: 사전과 오류 허용 검색

- 그림: 바이그램 색인 — BO → aboard, about, boardroom, border / OR → border, lord, morbid, sordid / RD → aboard, ardent, boardroom, border

✍ 메모

💡 이해 3강 k-gram 색인 (와일드카드·철자 교정)

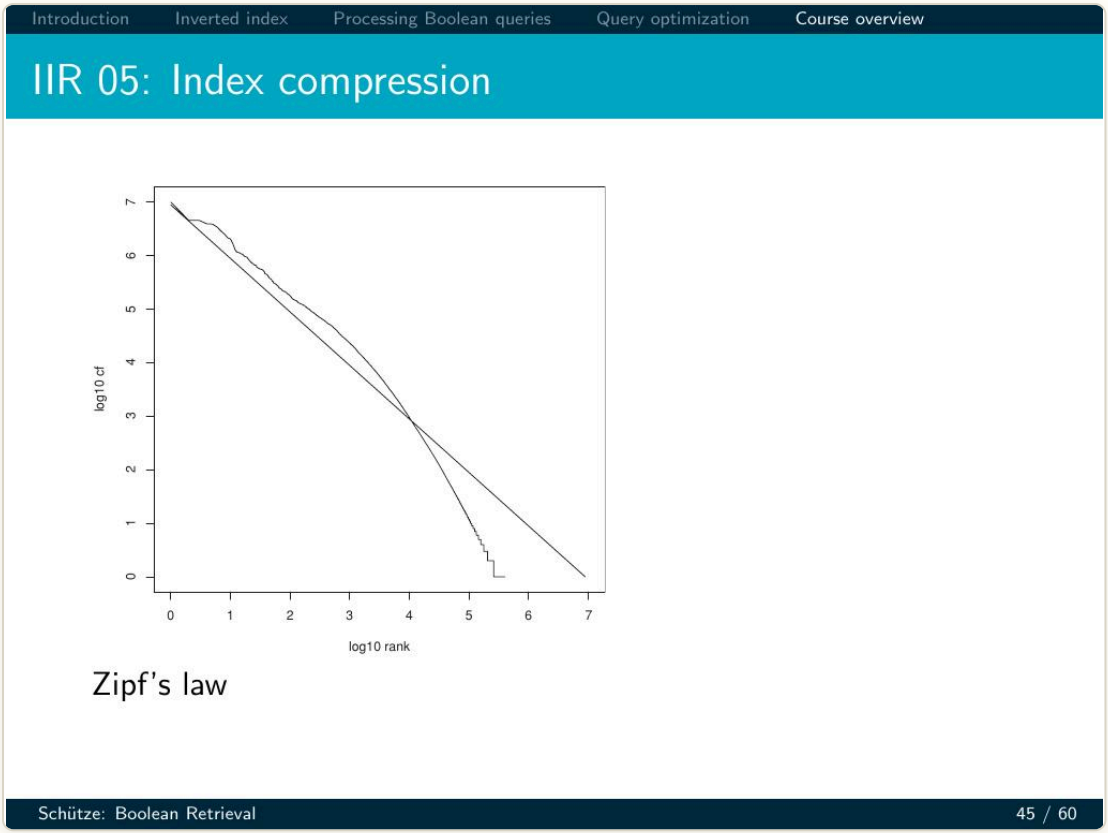


IIR 04: 색인 구축

- 그림: 분산 색인(MapReduce) — master가 splits를 parser에 배정 → a-f / g-p / q-z 세그먼트 파일 (**map 단계**) → inverter가 범위별로 모아 포스팅 생성 (**reduce 단계**)

✍️ 메모

💡 이해 4강

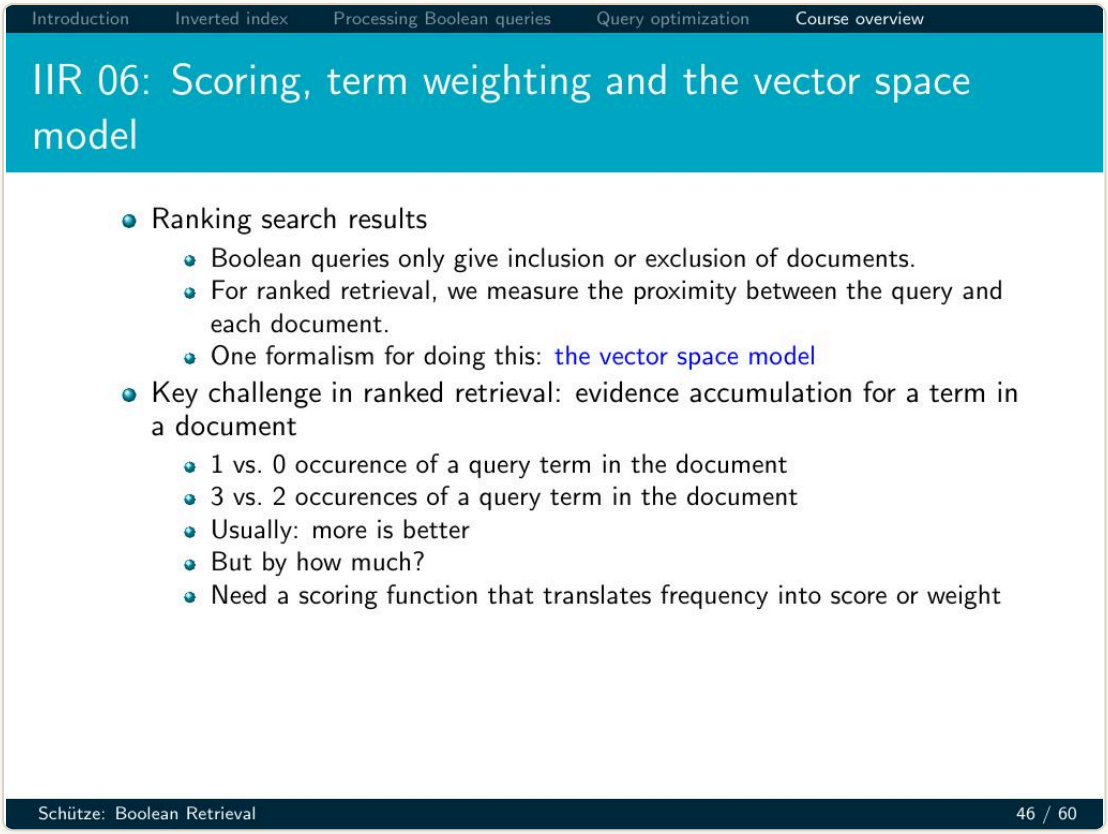


IIR 05: 색인 압축

- 그림: **지프의 법칙(Zipf's law)** — log10 순위 vs log10 컬렉션 빈도(cf)가 거의 직선

✍️ 메모

💡 이해 5강

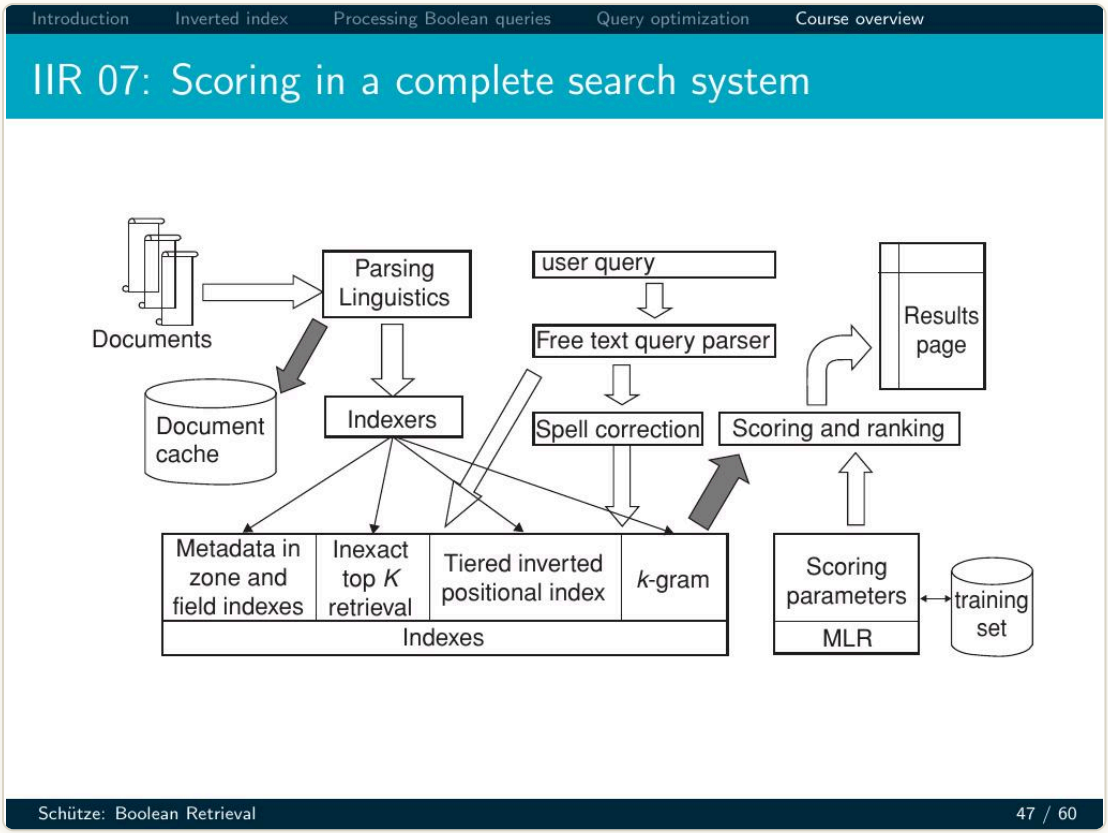


IIR 06: 점수화, 단어 가중치, 벡터 공간 모델

- **검색 결과 순위화**
  - 불리언 질의는 문서의 **포함/제외만** 알려 준다
  - 순위 검색에선 질의와 각 문서의 **가까움**을 잰다
  - 그 방법 하나: **벡터 공간 모델**
- 순위 검색의 핵심 과제: 문서 안 단어의 **증거 누적**
  - 질의어가 문서에 1번 vs 0번
  - 3번 vs 2번
  - 보통 **많을수록 좋다** — 그런데 **얼마나**?
- 빈도를 점수(가중치)로 바꿔 줄 **점수 함수**가 필요

✍️ 메모

💡 이해 6강 tf-idf로 이어짐

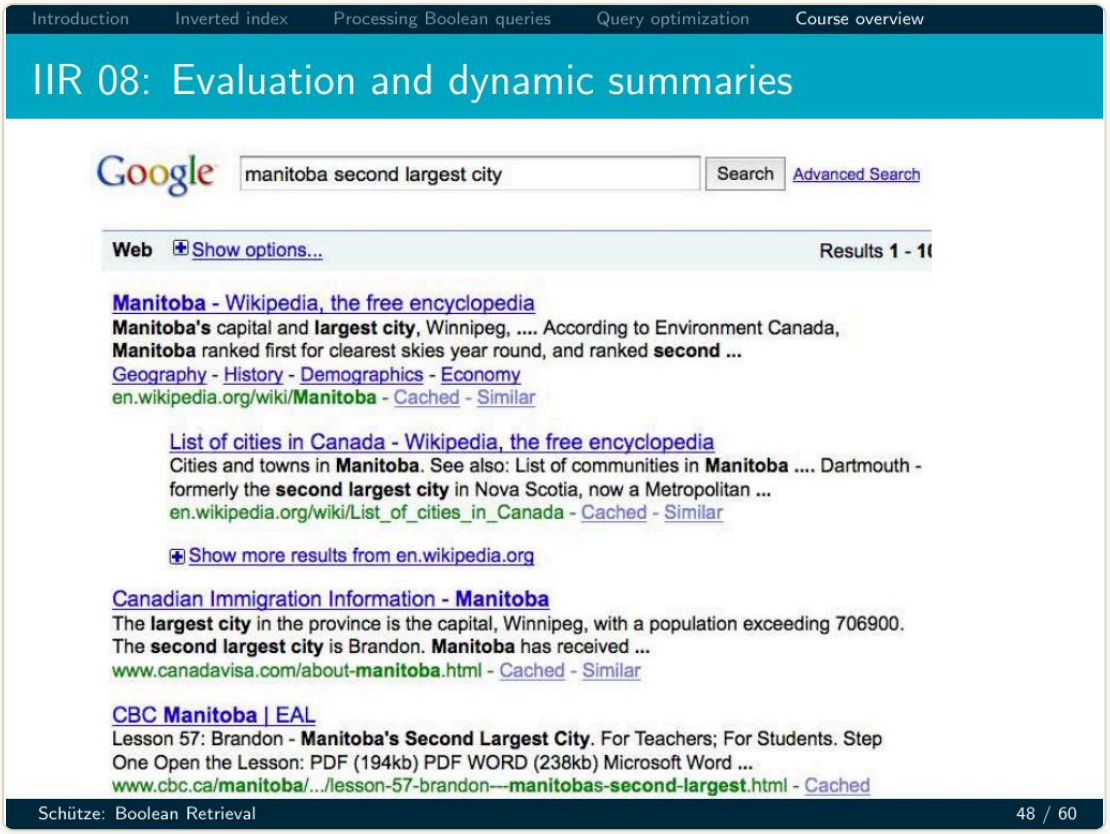


IIR 07: 완전한 검색 시스템에서의 점수화

- 그림: 문서 → 파싱·언어처리 → 색인(메타데이터·영역 색인, 부정확 top K, 계층 위치 색인, k-gram) / 사용자 질의 → 질의 파서 → 철자 교정 → 점수화·순위 → 결과 페이지, 점수 매개변수는 학습 집합으로 MLR(기계학습 순위)

✍️ 메모

💡 이해 7강

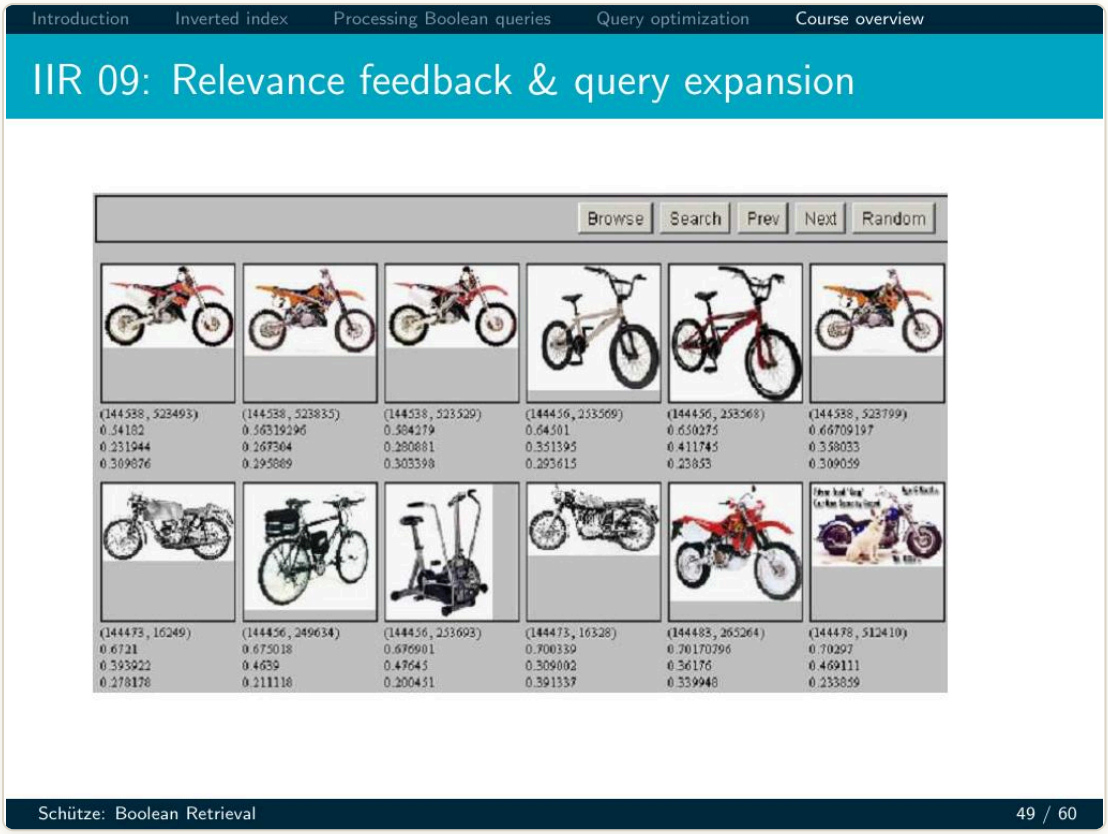


IIR 08: 평가와 동적 요약

- 그림: 구글 검색 결과 화면 — 질의 'manitoba second largest city'에 대한 결과와 **스니펫(요약)**

✍ 메모

— 넘어가도 되는 쪽



IIR 09: 적합성 피드백과 질의 확장

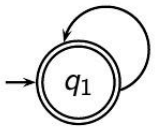
- 그림: 이미지 검색 결과 — 사용자가 관련 있는 그림을 고르면 그걸 반영해 다시 검색

✍ 메모

— 넘어가도 되는 쪽

Introduction Inverted index Processing Boolean queries Query optimization Course overview

IIR 12: Language models



w	$P(w\|q_1)$	w	$P(w\|q_1)$
STOP	0.2	toad	0.01
the	0.2	said	0.03
a	0.1	likes	0.02
frog	0.01	that	0.04
		...	...

This is a one-state probabilistic finite-state automaton – a unigram language model – and the state emission distribution for its one state q_1.

STOP is not a word, but a special symbol indicating that the automaton stops.

frog said that toad likes frog STOP

$P(\text{string}) = 0.01 \cdot 0.03 \cdot 0.04 \cdot 0.01 \cdot 0.02 \cdot 0.01 \cdot 0.2$
$= 0.0000000000048$

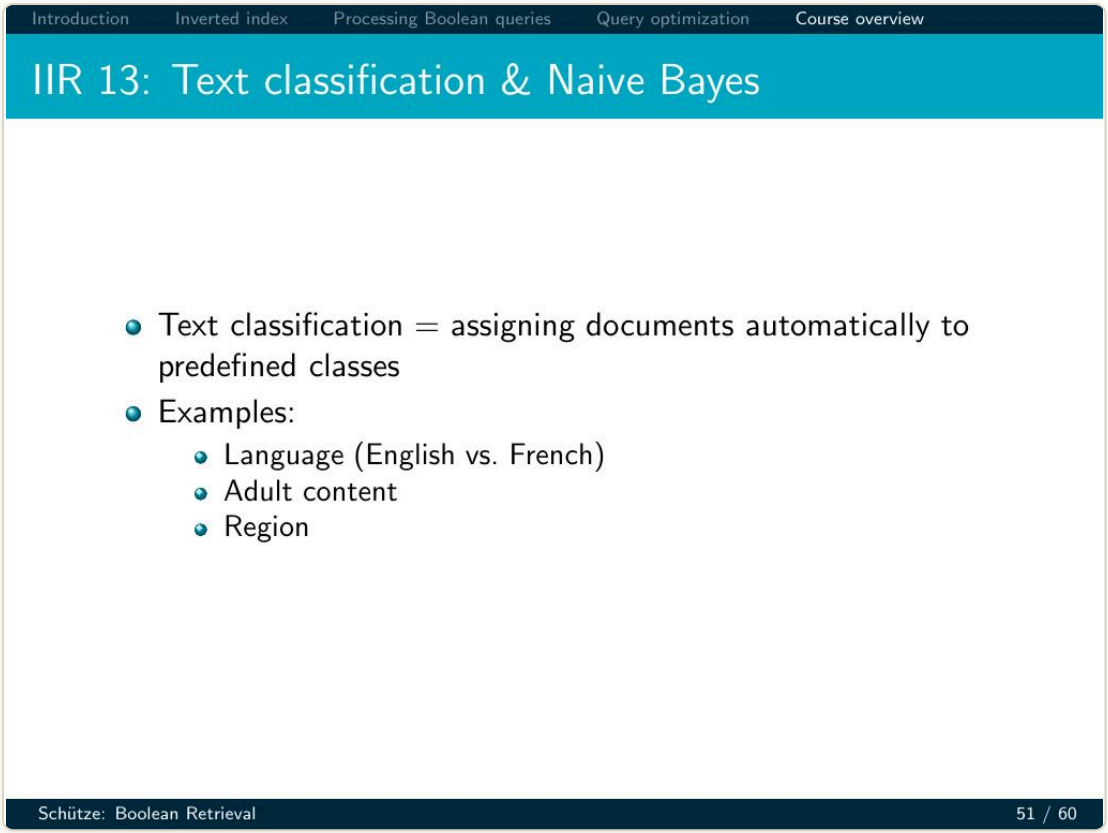
Schütze: Boolean Retrieval 50 / 60

IIR 12: 언어 모델

- 상태 하나짜리 확률적 유한 오토마톤 = **유니그램 언어 모델**, 상태 q1의 방출 분포
- STOP은 단어가 아니라 오토마톤이 멈춘다는 특수 기호
- 예: frog said that toad likes frog STOP
  - P = 0.01 · 0.03 · 0.04 · 0.01 · 0.02 · 0.01 · 0.2 = 0.0000000000048

✍ 메모

— 넘어가도 되는 쪽

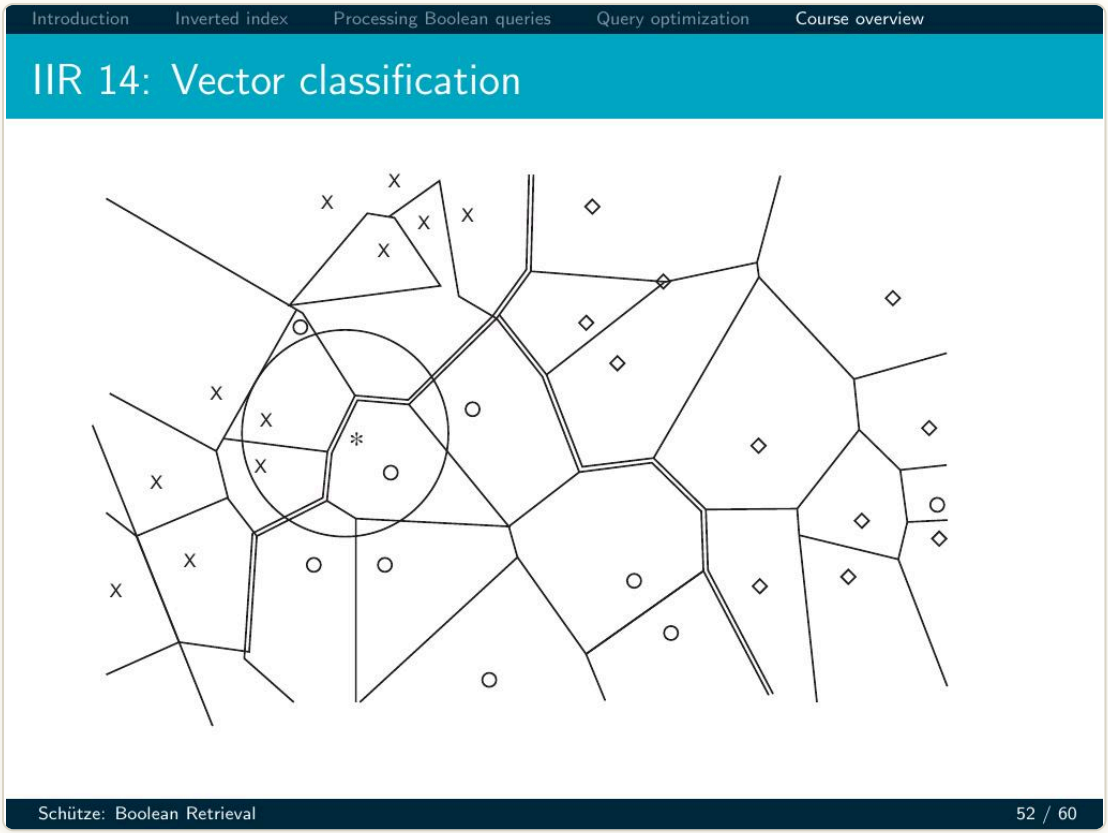


IIR 13: 텍스트 분류와 나이브 베이즈

- 텍스트 분류 = 문서를 **미리 정한 부류**에 자동으로 배정
- 예: 언어(영어 vs 프랑스어), 성인물, 지역 ...

✍️ 메모

— 넘어가도 되는 쪽

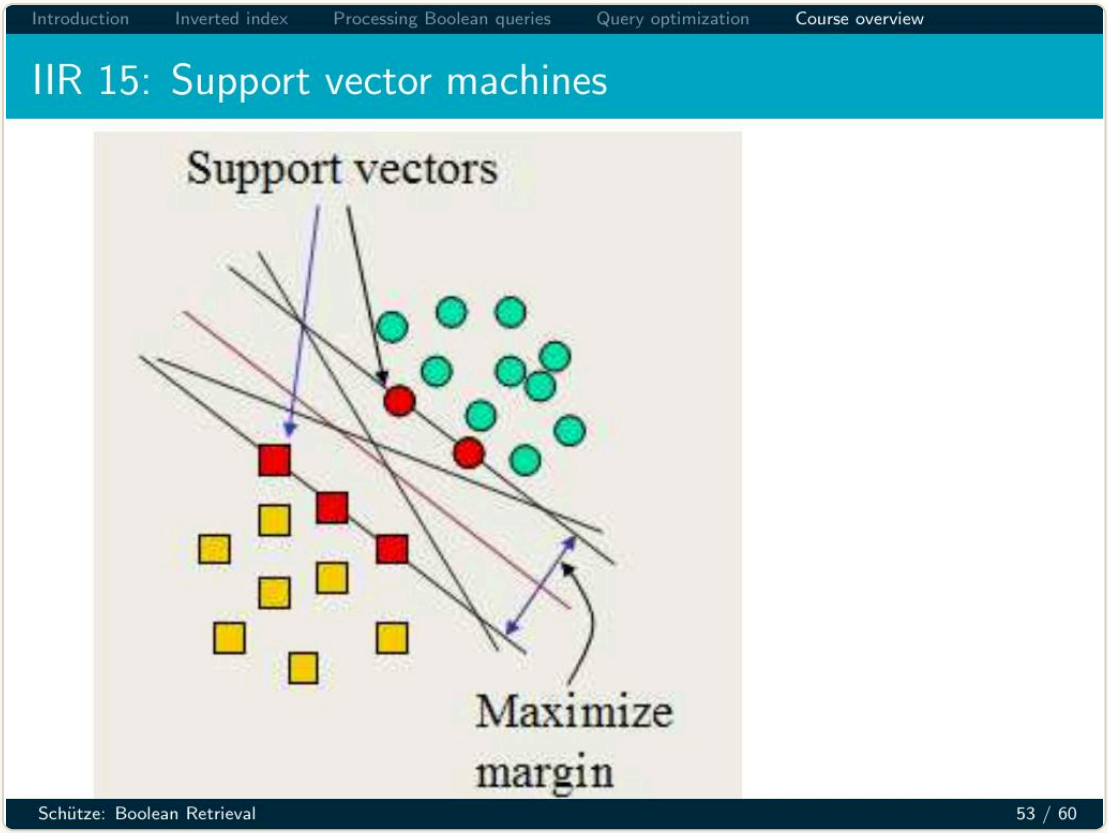


IIR 14: 벡터 분류

- 그림: 보로노이 분할 — kNN 분류의 결정 경계

✍ 메모

— 넘어가도 되는 쪽

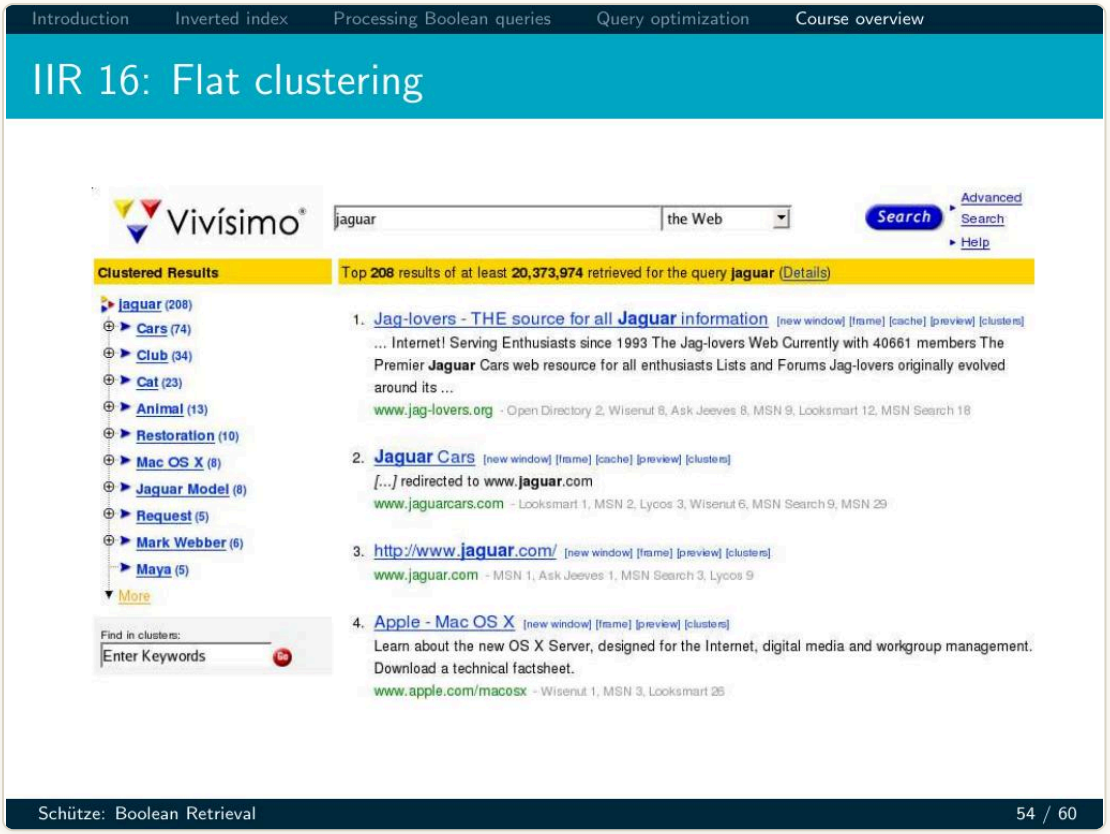


IIR 15: 서포트 벡터 머신

- 그림: 두 부류 사이 **마진을 최대화**하는 경계와 서포트 벡터

✍ 메모

— 넘어가도 되는 쪽



IIR 16: 평면 군집화

- 그림: Vivísimo — 'jaguar' 검색 결과를 자동차·동물·Mac OS X 등 **군집**으로 묶어 보여 줌

✍ 메모

— 넘어가도 되는 쪽

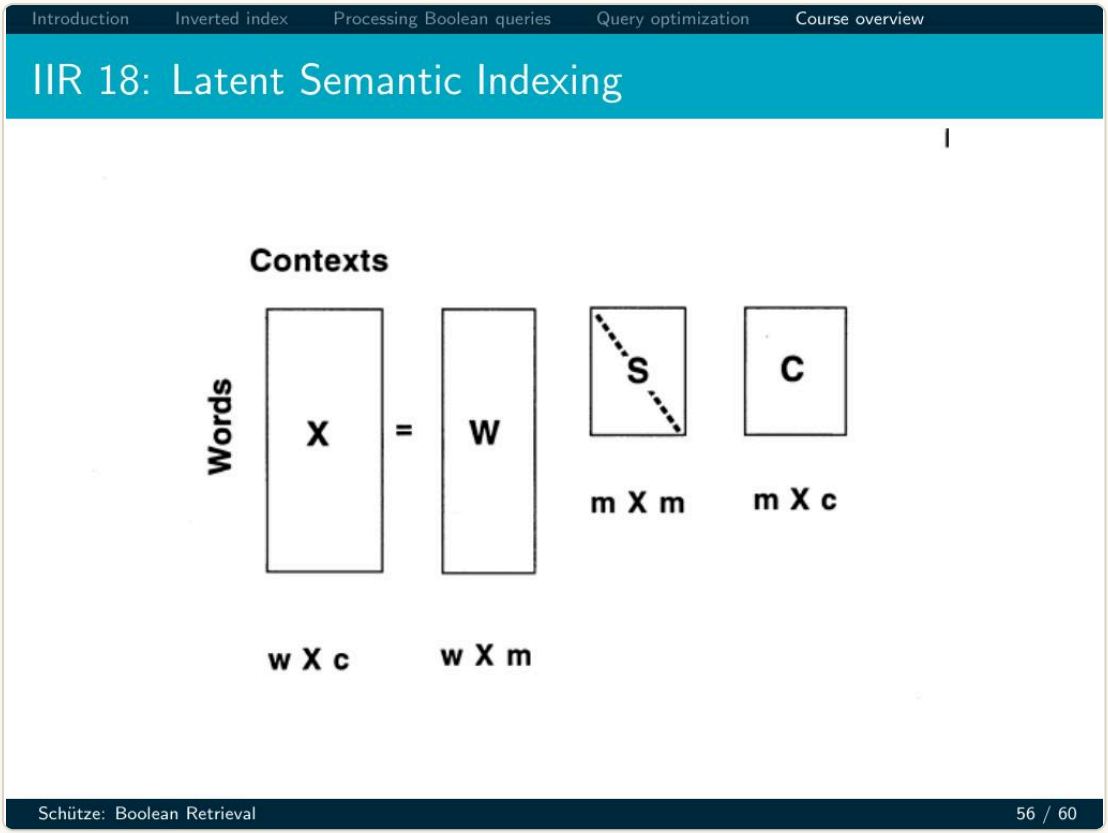


IIR 17: 계층 군집화

- 그림: 구글 뉴스 — 비슷한 기사를 묶은 화면

✍ 메모

— 넘어가도 되는 쪽

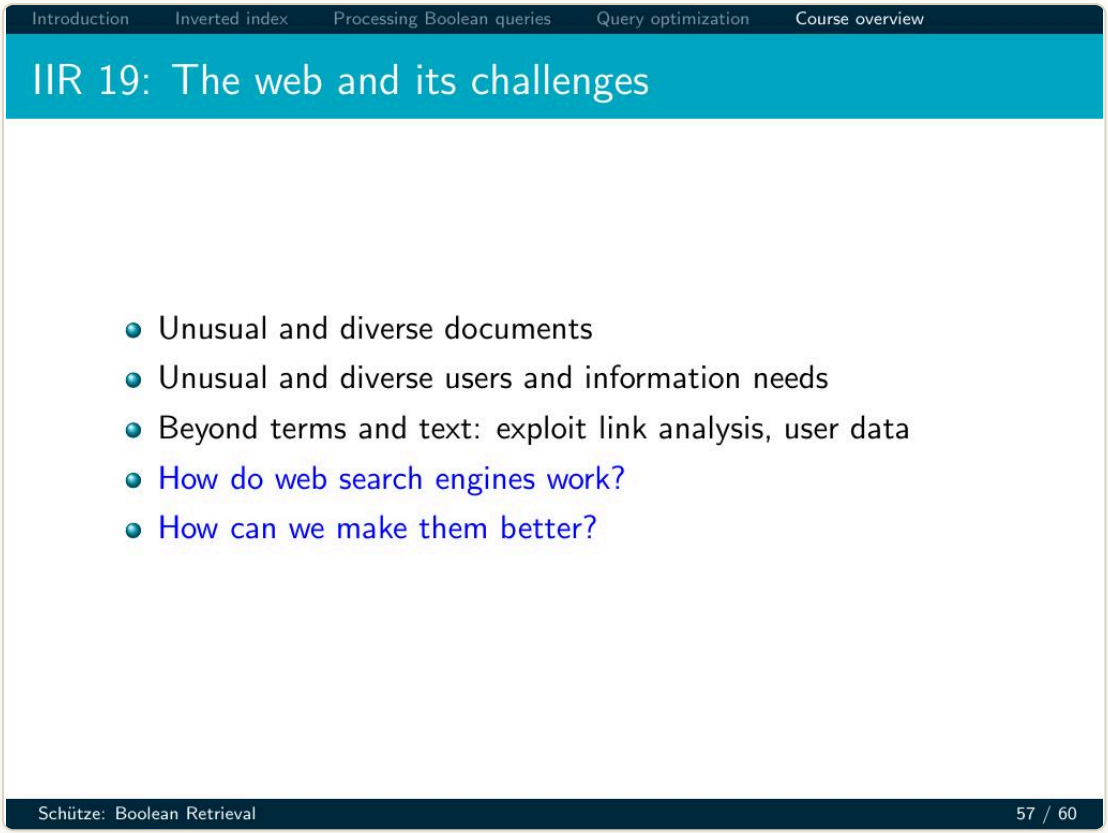


IIR 18: 잠재 의미 색인 (LSI)

- 그림: 단어-문서 행렬의 저차원 근사

✍ 메모

— 넘어가도 되는 쪽

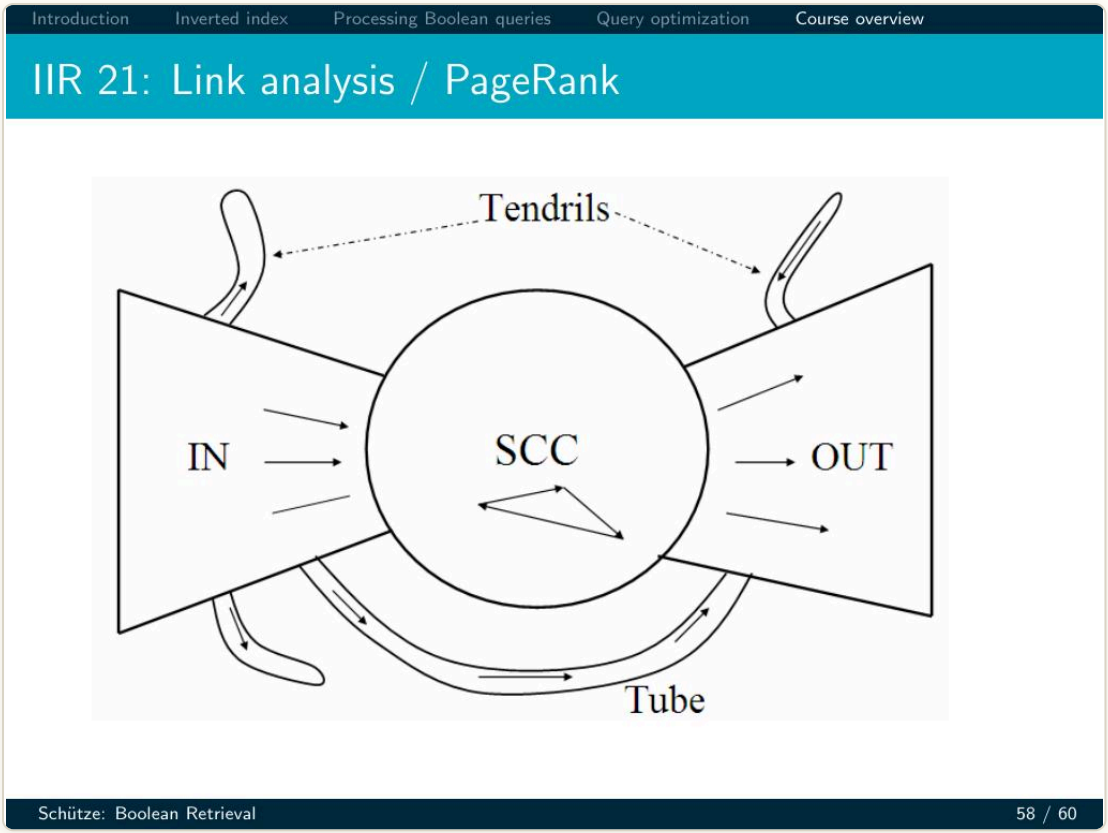


IIR 19: 웹과 그 도전

- 특이하고 다양한 **문서**
- 특이하고 다양한 **사용자와 정보 요구**
- 단어와 텍스트 너머: **링크 분석, 사용자 데이터** 활용
- 웹 검색 엔진은 어떻게 동작하나? 어떻게 더 좋게 만들까?

✍️ 메모

— 넘어가도 되는 쪽

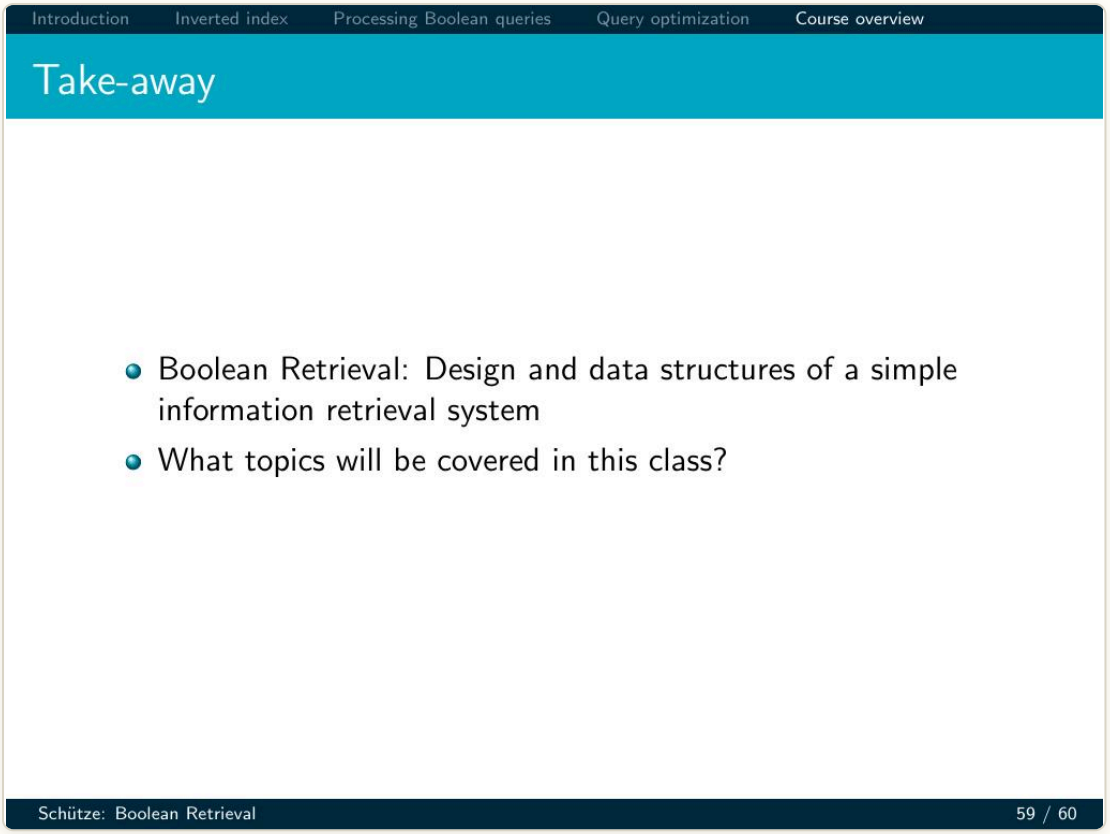


IIR 21: 링크 분석 / PageRank

- 그림: 웹 그래프와 PageRank

✍ 메모

— 넘어가도 되는 쪽

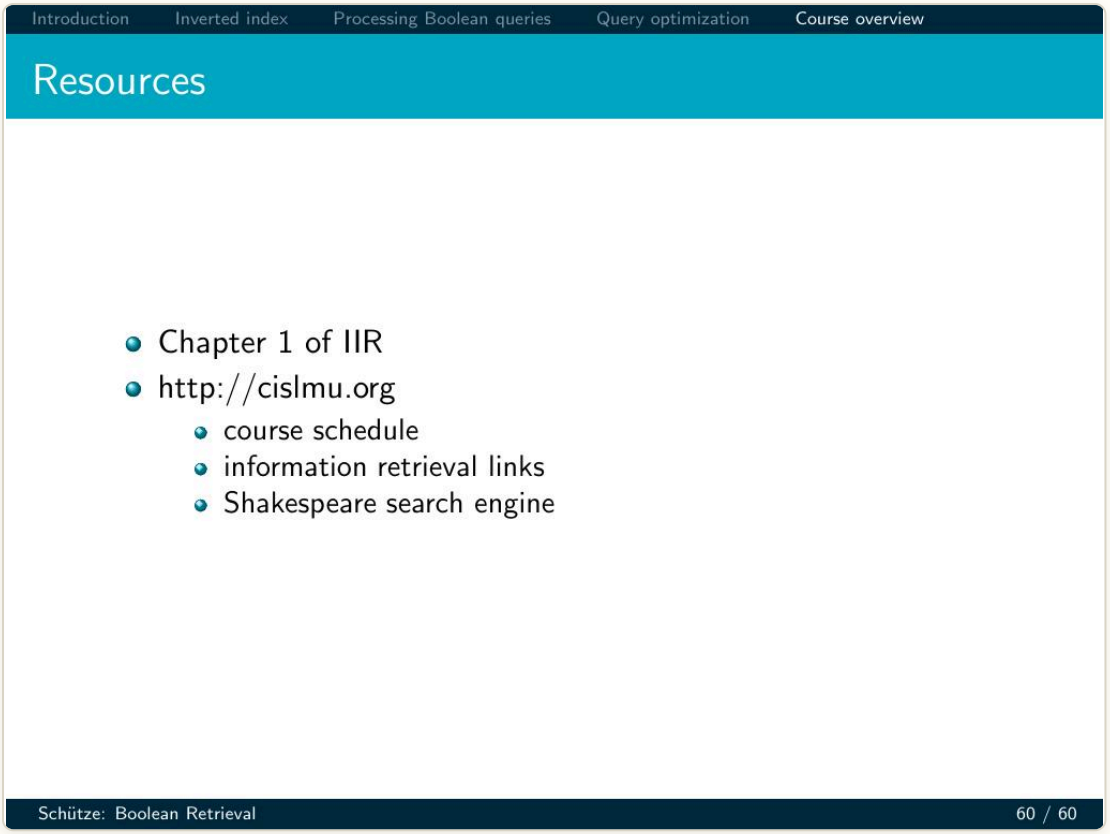


이번 강의에서 가져갈 것

- 불리언 검색: 간단한 IR 시스템의 설계와 자료구조 (발생 행렬 → 역색인 → 교집합 → 최적화)
- 이 수업에서 다룰 주제

✍️ 메모

— 넘어가도 되는 쪽



참고 자료

- IIR 1장
- http://cislmu.org — 강의 일정, IR 링크, 셰익스피어 검색 엔진

✍️ 메모

— 넘어가도 되는 쪽